Vehicle Optimal Design Lab Report

Alessandro Ferrario (10766865)
Andrea Sgotto (10796906)
Giacomo Sanguin (10732325)

a.y. 2025 / 2026

Contents

1 Lab 1: Braking system optimization **1**
- 1.1 Problem Definition . . . 1
- 1.2 Matlab implementation . . . 1
- 1.3 Conclusions . . . 3

2 Lab 2: Passive suspension optimization (sorting algorithm) **5**
- 2.1 Problem Definition . . . 5
- 2.2 Matlab implementation . . . 6
  - 2.2.1 Case 1: Pareto-optimal set for discomfort and road holding . . . 7
  - 2.2.2 Case 2: Pareto-optimal set for discomfort, road holding and working space . . . 8

3 Lab 3: Passive suspension optimization (weighted sum, constraint method and analytical derivation of PO set) **11**
- 3.1 Problem Definition . . . 11
- 3.2 Matlab implementation . . . 11
  - 3.2.1 Weighted sum . . . 11
  - 3.2.2 Constraint method . . . 13
  - 3.2.3 Analytical derivation of the Pareto-optimal set . . . 16

4 Lab 4: Active suspension optimization **19**
- 4.1 Problem Definition . . . 19
- 4.2 Matlab implementation . . . 20
- 4.3 Comparison with passive suspension results . . . 21

5 Lab 6: Genetic Algorithms (Optimization of a Paraboloid) **22**
- 5.1 Problem Definition . . . 22
- 5.2 Matlab Implementation . . . 22
  - 5.2.1 Case 1: Standard GA . . . 23
  - 5.2.2 Case 2: Standard GA:mutation added . . . 25
  - 5.2.3 Case 3: Standard GA:elitism strategy . . . 27
- 5.3 Comparing results . . . 29

6 Lab 7: GA Multi-objective optimization **30**
- 6.1 Double wishbone suspension design . . . 30
- 6.2 Optimization Methodology . . . 31
  - 6.2.1 Fitness Assignment via Pareto Ranking . . . 31
- 6.3 Simulation Results and Comparative Analysis . . . 32

7 Optimization via Adams Insight **35**
- 7.1 Balancer . . . 35
  - 7.1.1 Design Variable . . . 35
  - 7.1.2 Objective Function . . . 35
  - 7.1.3 Optimization Strategy: DOE and Adams Insight . . . 36
  - 7.1.4 Results and Comparative Analysis . . . 37
- 7.2 Case Study 2: Retractable Landing Gear Optimization . . . 38
  - 7.2.1 System Overview . . . 38

7.2.2 Design Variables . . . 38
7.2.3 Optimization Objectives . . . 38
7.2.4 Preliminary simulations . . . 39
7.2.5 Adams Insight . . . 40
7.3 Ferrari Testarossa Suspension Analysis (Adams Car) . . . 41
7.3.1 Parallel Wheel Travel Simulation . . . 41
7.3.2 DOE Setup in Adams Insight . . . 41
7.3.3 Results . . . 42
7.4 Full Vehicle Analysis: Step Steer Optimization . . . 43
7.4.1 Optimization Setup . . . 43
7.4.2 Objectives . . . 43
7.4.3 DOE Methodology and Results . . . 44

8 Lab 8: Neural Network Application on a 2-Variables Function 45
8.1 Problem definition: . . . 45
8.2 Research on Network Parameters influence: . . . 46
8.2.1 Network Capacity (Number of Neurons): . . . 46
8.2.2 Architecture Efficiency (Number of total Weights): . . . 47
8.2.3 Training algorithms (Standard vs Regularization): . . . 48
8.3 Final Network Design: . . . 49

9 Lab 9: Vehicle Handling Optimization 51
9.1 Problem definition: . . . 51
9.2 Design Variables and Objective Functions: . . . 51
9.3 Correlation Matrix: . . . 52
9.4 Neural Network Training: . . . 53
9.5 Optimization Procedure: . . . 55
9.5.1 Pareto Optimal Set: . . . 55
9.5.2 Choice of the Best Solution: . . . 56
9.5.3 Simulation: . . . 57

10 Lab 11: Structural Optimization Laboratory 59
10.1 Matlab 99 line optimization code . . . 59
10.1.1 Problem definition: . . . 59
10.1.2 Experimental Procedure: . . . 59
10.1.3 Results and comments: . . . 60
10.2 Lower control arm Optimization on Inspire . . . 65
10.2.1 Problem definition: . . . 65
10.2.2 Optimization Setup: . . . 65
10.2.3 Results: . . . 66

11 Lab RL: Suspension Optimization (Reinforcement learning) 68
11.1 Problem definition . . . 68
11.2 Reinforcement Learning implementation: . . . 68
11.2.1 Working Principle: . . . 68
11.3 Reward criteria implementation . . . 69
11.4 Analysis of RL Optimization Result: . . . 70

1 Lab 1: Braking system optimization

1.1 Problem Definition

The aim of this laboratory is to optimize the braking force distribution of a car. Specifically, the real braking distribution must be as close as possible to the ideal one to maximize friction and reduce stopping distance. However, it must not lie above the ideal distribution curve; otherwise, instability would be generated due to the rear wheels locking before the front ones.

The real braking force distribution is given by the equation:

$$X_R = X_F \left(\frac{D_R}{D_F}\right)^2 \tag{1}$$

- X_R: rear braking force [N];
- X_F: front braking force [N];
- D_R: piston diameter of rear brake caliper [mm];
- D_F: piston diameter of front brake caliper [mm];

Given the ideal rear braking force $X_{R,id}$, the optimization is achieved by minimizing the Mean Squared Error (MSE) between the ideal and the real rear braking forces, considering the piston diameters D_F and D_R as design variables. The diameters are constrained between 15 mm and 50 mm.

$$E = \frac{1}{N}\sum (X_{R,id} - X_R)^2 \tag{2}$$

- E: Mean Squared Error [N^2];

Objective

$$\min E(d) \quad \text{with} \quad d = [D_F \;\; D_R] \in \Omega \subset \mathbb{R}^2$$
$$\Omega = [15 \;\; 50] \times [15 \;\; 50]$$

1.2 Matlab implementation

First, the ideal braking force distribution is computed by creating arrays for $X_{F,id}$ and $X_{R,id}$ according to the following equations:

$$X_{F,id} = m\ddot{x}\left(\frac{b}{a+b} + \frac{\ddot{x}}{g}\frac{h_G}{a+b}\right) \tag{3}$$

$$X_{R,id} = m\ddot{x}\left(\frac{a}{a+b} - \frac{\ddot{x}}{g}\frac{h_G}{a+b}\right) \tag{4}$$

- m: vehicle mass [kg];
- $\ddot{x}$: vehicle deceleration [$\frac{m}{s^2}$];
- g: gravitational acceleration [9.80665 $\frac{m}{s^2}$];

- a: front wheelbase [m];
- b: rear wheelbase [m];
- h_G: height of the center of mass [m];

The parameter values are shown in the following table:

Parameter	**Value**
Mass m	1127 kg
Wheelbase $a + b$	2.57 m
Weight distribution	53% front 47% rear
Center of mass height h_G	0.533 m

Table 1: Vehicle characteristics

$\ddot{x}$ is defined as an array ranging from 0 to μg, where μ is the friction coefficient between the tires and the ground ($\mu = 0.9$). The resulting ideal distribution is a curve.

Subsequently, the real braking force distribution is modeled as $X_R = X_R(X_{F,id})$. Since the real distribution is an inclined straight line that could exceed the ideal curve (causing stability issues), the force value at the intersection between the real and ideal distributions is kept constant. This behavior represents the operation of a braking force proportioning valve that adjusts the pressure sent to the rear brakes.

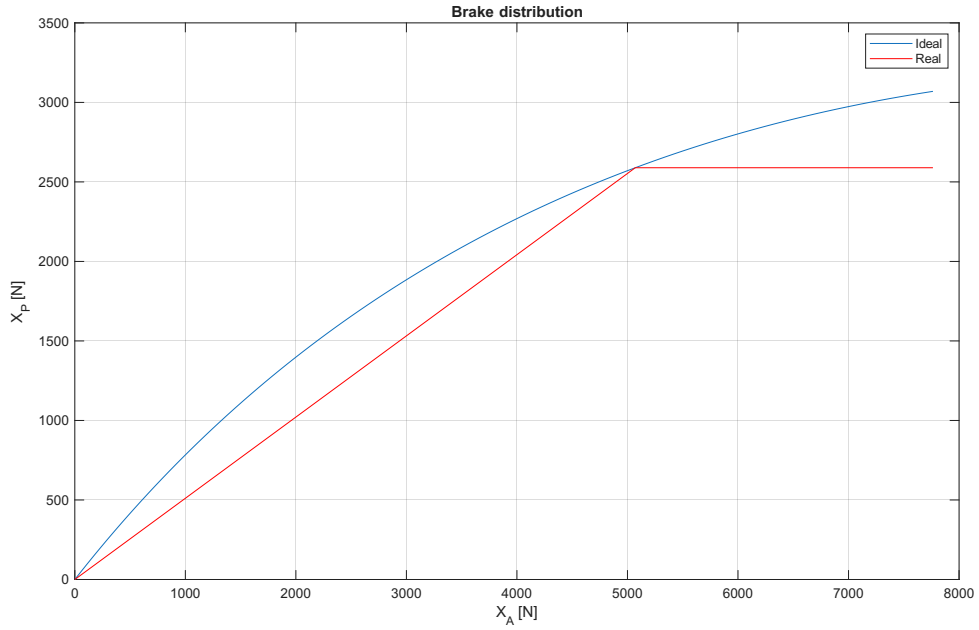


Figure 1: Ideal and real braking force distributions

Then, the mean squared error is computed through a customized implemented Matlab function `msqerr`. The function takes the piston diameters (contained in the variable `x`) and the ideal braking forces as input and provides the error `e` as output.

```
function e =  msqerr(x,Xa_id,Xp_id)
        if any(x<15 )||any(x>50)
            e = 1e9;
            return
        end
         valve = true;
         Xp = zeros(length(Xp_id),1);
         for i=1:length(Xp_id)
             if valve
                Xp(i) = Xa_id(i) * (x(2)/x(1))^2;
             end
             if Xp(i) > Xp_id(i) && valve
                 valve = false;
                 X_pclose = Xp_id(i-1);
             end
             if not(valve)
                Xp(i)= X_pclose;
             end
         end
         e = 1/length(Xp_id)*sum((Xp_id-Xp).^2);
end
```

As shown, a penalty (`e = 1e9`) is added to the code to solve the optimization problem within the constrained set of diameters. The function calculates the error while accounting for the activation of the braking force distributor.

The piston diameters that minimize the error are computed using the Matlab function `fminsearch` (simplex method). Starting from an initial set of diameters `x0`, the function modifies the values until the distribution with the minimum MSE is obtained.

```
x0=[40 , 40]; % initial vector for initialization, x = [df , dr]
x=fminsearch(@(x) msqerr(x,Xa_id,Xp_id),x0);
```

1.3 Conclusions

Different solutions can be obtained by varying the initial point `x0`. In fact, this problem configuration yields infinite solutions; many sets of diameters provide a braking distribution with the same minimum mean squared error, as shown in Figure 2.

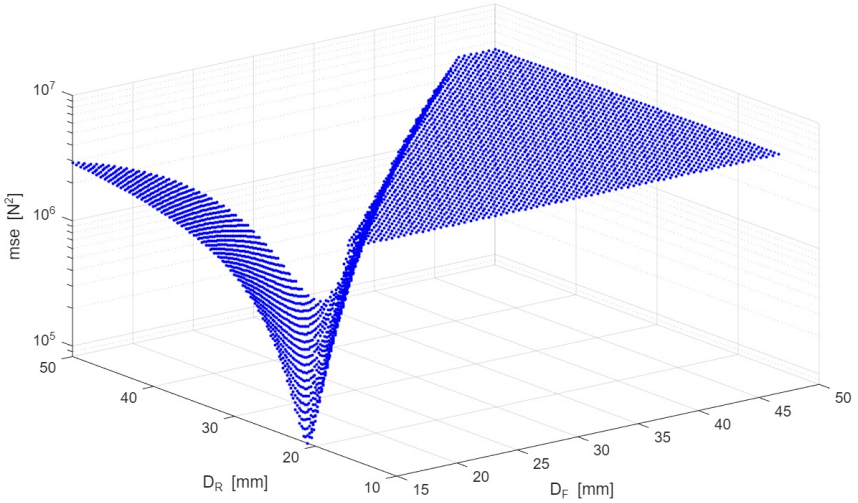


Figure 2: Set of piston diameters with relative mean squared error

Therefore, the problem is not well-posed because it lacks a unique solution. However, it was observed that all possible solutions are characterized by the same diameter ratio D_F/D_R. Based on this, the problem can be reformulated considering only the diameter ratio as a design variable. In this case, the problem becomes strictly convex, resulting in a unique solution with the minimum MSE.

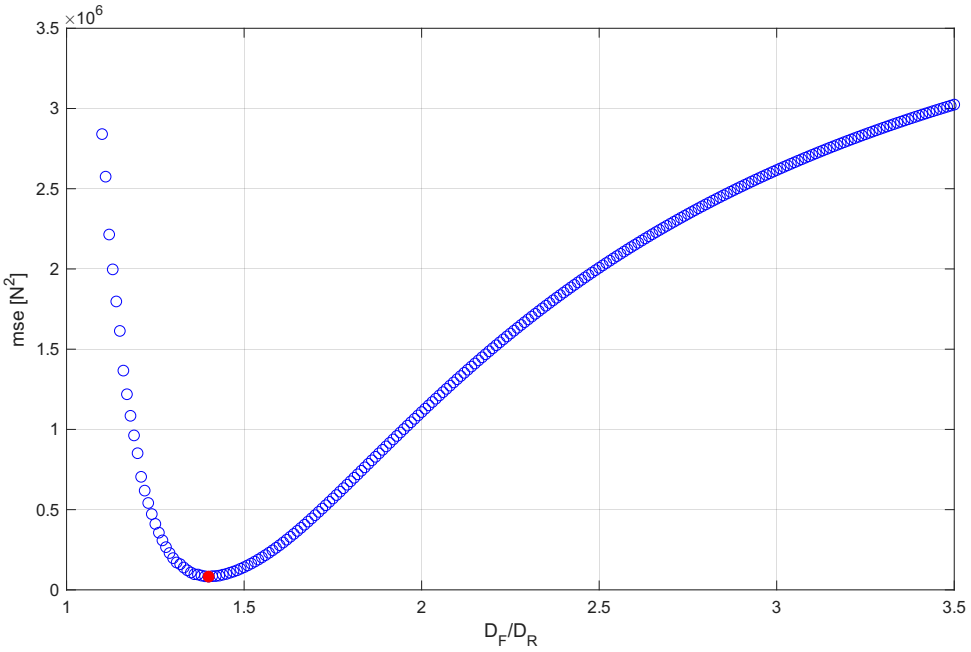


Figure 3: Diameter ratios with relative mean squared error

The final solution is represented by the red dot in the figure.

2 Lab 2: Passive suspension optimization (sorting algorithm)

2.1 Problem Definition

The aim of this laboratory is to optimize the passive suspension system of a car, based on a 2-DoF (Degrees of Freedom) quarter-car vehicle model.

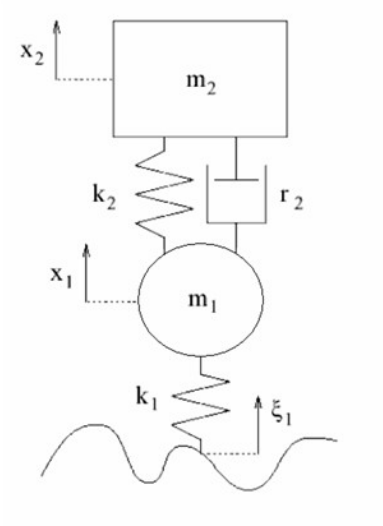


Figure 4: Quarter car vehicle model

The parameters are:

- m_1: unsprung mass [kg];
- m_2: sprung mass [kg];
- k_1: tyre radial stiffness [N/m];
- k_2: suspension equivalent stiffness (assumed linear) [N/m];
- r_2: suspension equivalent damping (assumed linear) [Ns/m];
- ξ_1: road irregularity [m];

Specifically, the goal is to compute the Pareto-optimal set of solutions by adjusting the suspension equivalent stiffness k_2 and the suspension equivalent damping r_2, which are the design variables of the problem.

The objective functions to be minimized are: the discomfort $\sigma_{\ddot{x}_2}$, the road holding σ_{F_z}, and the working space $\sigma_{x_2-x_1}$.

Discomfort $\sigma_{\ddot{x}_2}$ is defined as the standard deviation of the vertical acceleration in the cabin; road holding σ_{F_z} is the standard deviation of the vertical force acting on the tyre; and working space $\sigma_{x_2-x_1}$ is the standard deviation of the relative displacement between the sprung and unsprung masses.

All the objective functions are represented by the following equations:

$$\sigma_{\ddot{x}_2} = A \cdot \sqrt{\frac{(m_1 + m_2)}{m_2^2 r_2} k_2^2 + \frac{k_1 r_2}{m_2^2}} \tag{5}$$

$$\sigma_{F_z} = A \cdot \sqrt{\frac{(m_1 + m_2)^3}{m_2^2 r_2} k_2^2 - 2\frac{m_1 k_1 (m_1 + m_2)}{m_2 r_2} k_2 + \frac{k_1 r_2 (m_1 + m_2)^2}{m_2^2} + \frac{k_1^2 m_1}{r_2}} \tag{6}$$

$$\sigma_{x_2 - x_1} = A \cdot \sqrt{\frac{m_1 + m_2}{r_2}} \tag{7}$$

with

$$A = \sqrt{\frac{A_b v}{2}} \tag{8}$$

- A_b: road roughness coefficient [m];
- v: vehicle velocity [m/s];
- A: noise intensity parameter [m/$\sqrt{s}$].

Reference values for the parameters are reported in Table 2.

Parameter	**Value**
Road roughness coefficient A_b	1.4e-5 m
Velocity v	30 m/s
Sprung mass m_2	230 kg
Unsprung mass m_1	30 kg
Tyre radial stiffness k_1	120,000 N/m

Table 2: Parameters of the problem

The design variable k_2 varies between 0 N/m and 50,000 N/m, while r_2 varies between 0 Ns/m and 5,000 Ns/m.

2.2 Matlab implementation

The Pareto-optimal set was computed by applying a sorting algorithm starting from a uniform grid of the design variables k_2 and r_2. The grid was generated according to the following equation:

$$n_{cdv} = n_v^{n_{dv}} \tag{9}$$

- n_{cdv}: number of possible combinations of design variable values;
- n_v: number of values per design variable;
- n_{dv}: number of design variables.

The Matlab function `combvec` was used to build the matrix of all possible combinations. Given 100 values for each design variable, the result is a $10,000 \times 2$ matrix called `DV`.

```
k2 = linspace(0,50000,100); % suspension equivalent stiffness [N/m]
r2 = linspace(0,5000,100); % suspension equivalent damping [Ns/m]
DV = combvec(k2,r2)'; % table of possible solutions
```

The problem was divided into two cases. The first case considers two objective functions (discomfort and road holding), while the second one considers all three objective functions.

2.2.1 Case 1: Pareto-optimal set for discomfort and road holding

For each combination of design variables, the corresponding objective function values were calculated. The resulting vectors `sigma_x2pp` and `sigma_Fz` were combined with the design variable matrix `DV` and a column of zeros used as flags for the sorting algorithm. Non-physical results (NaN) were removed, and the entire matrix was sorted by `sigma_x2pp` in ascending order.

```
flags = zeros(size(DV,1),1); % flags for pareto algorithm
mat = [DV sigma_x2pp sigma_Fz flags];
n = isnan(mat(:,4));
index = n == 1;
mat(index,:) = [];
mat_sorted = sortrows(mat,3); % sorted matrix containing k2, r2, 2 ofs
    and flags
```

Then, the sorting algorithm was implemented. It evaluates all values in `mat_sorted` and identifies valid solutions by applying the definition of a Pareto-optimal solution. If a solution is dominated (not Pareto-optimal), its flag is changed from 0 to 1. Finally, a new matrix called `pareto_mat` was created containing only the Pareto-optimal set.

```
for a = 1:size(mat_sorted,1)
    for j = a:size(mat_sorted,1)
        if mat_sorted(j,4) > mat_sorted(a,4)
            mat_sorted(j,5) = 1;
        end
    end
end % Pareto algorithm
pareto_mat = mat_sorted;
index2 = pareto_mat(:,5) == 1;
pareto_mat(index2,:) = []; % Pareto-optimal solutions matrix
```

The Pareto-optimal set was computed in both the objective function space and the design variable space.

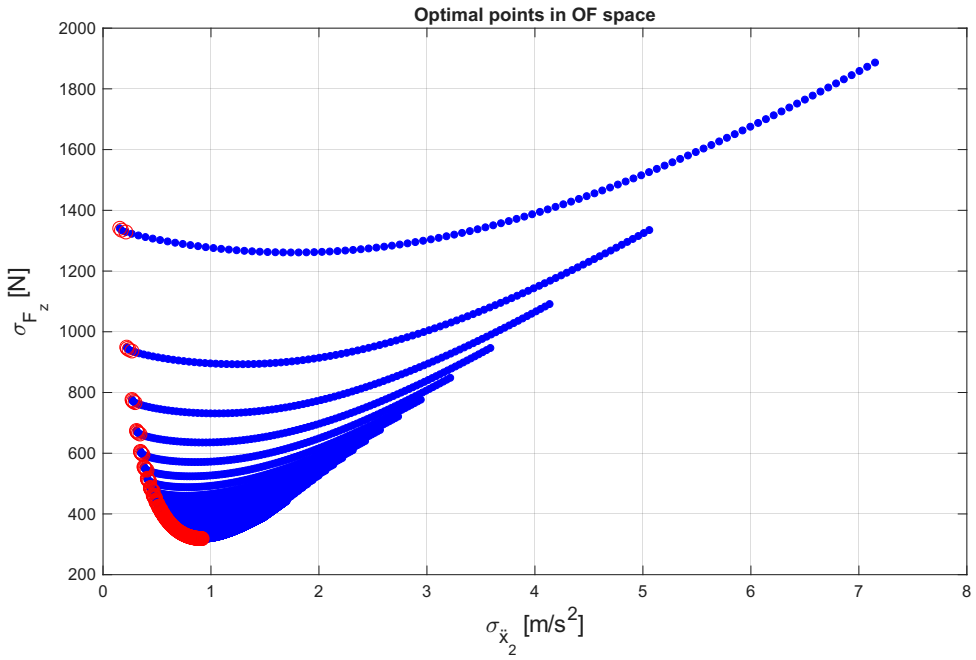


Figure 5: Pareto-optimal set in the space of the objective functions (Case 1)

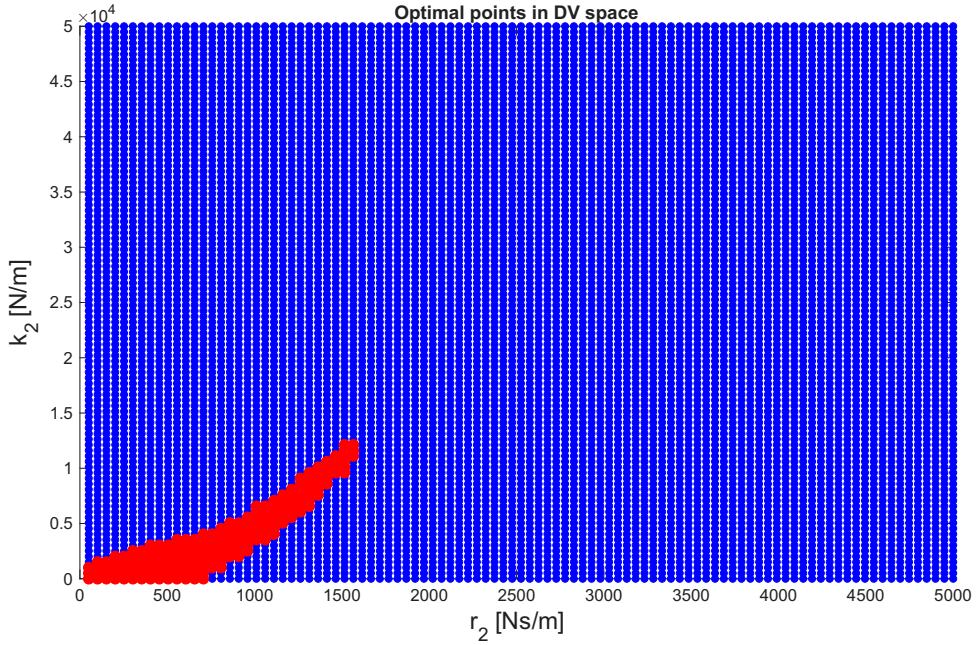


Figure 6: Pareto-optimal set in the space of the design variables (Case 1)

2.2.2 Case 2: Pareto-optimal set for discomfort, road holding and working space

The same approach as Case 1 was followed. However, the sorting algorithm was modified to adapt to the definition of Pareto-optimality for problems with more than two objective functions.

```
mat3D = [DV sigma_x2pp sigma_Fz sigma_x2_minus_x1 flag];
n2 = isnan(mat3D(:,4));
index3 = n2 == 1;
mat3D(index3,:) = [];
mat_sorted3D=sortrows(mat3D,3); % matrix containing k2, r2, 3 ofs and
    flags
for a = 1:size(mat_sorted3D,1)
   for j = a:size(mat_sorted3D,1)
        if mat_sorted3D(j,4) >= mat_sorted3D(a,4) && ...
   mat_sorted3D(j,5) >= mat_sorted3D(a,5) && ...
  (mat_sorted3D(j,4) >  mat_sorted3D(a,4) || ...
   mat_sorted3D(j,5) >  mat_sorted3D(a,5))
            mat_sorted3D(j,6) = 1;
        end
    end
end % Pareto algorithm
pareto_mat3D = mat_sorted3D;
index3 = pareto_mat3D(:,6) == 1;
pareto_mat3D(index3,:) = []; % Pareto-optimal solutions matrix
```

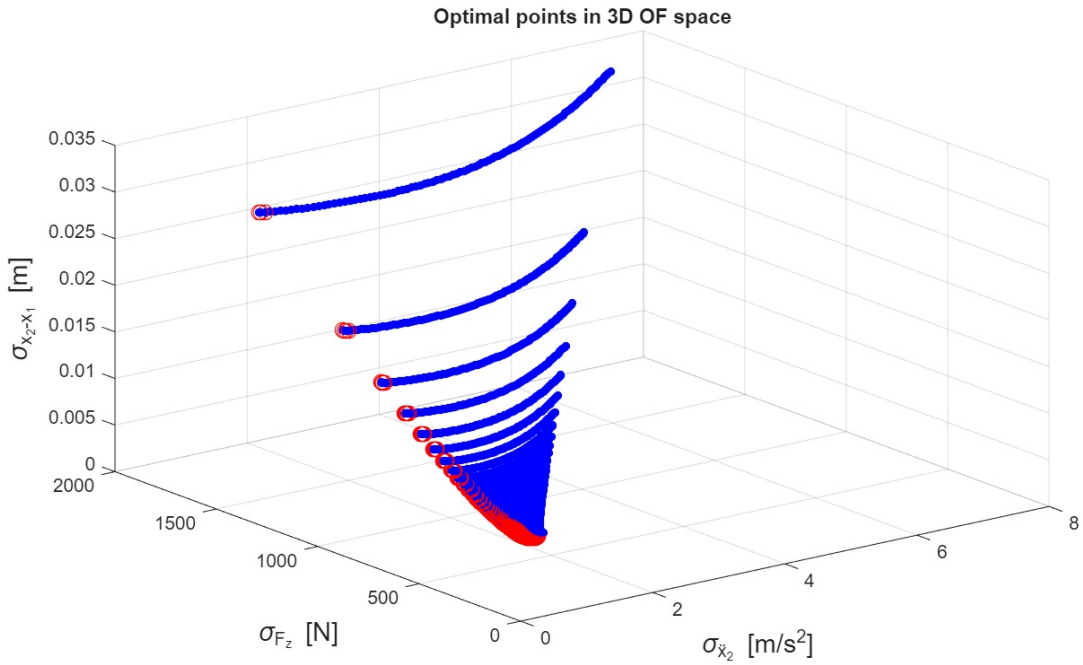


Figure 7: Pareto-optimal set in the space of the objective functions (Case 2)

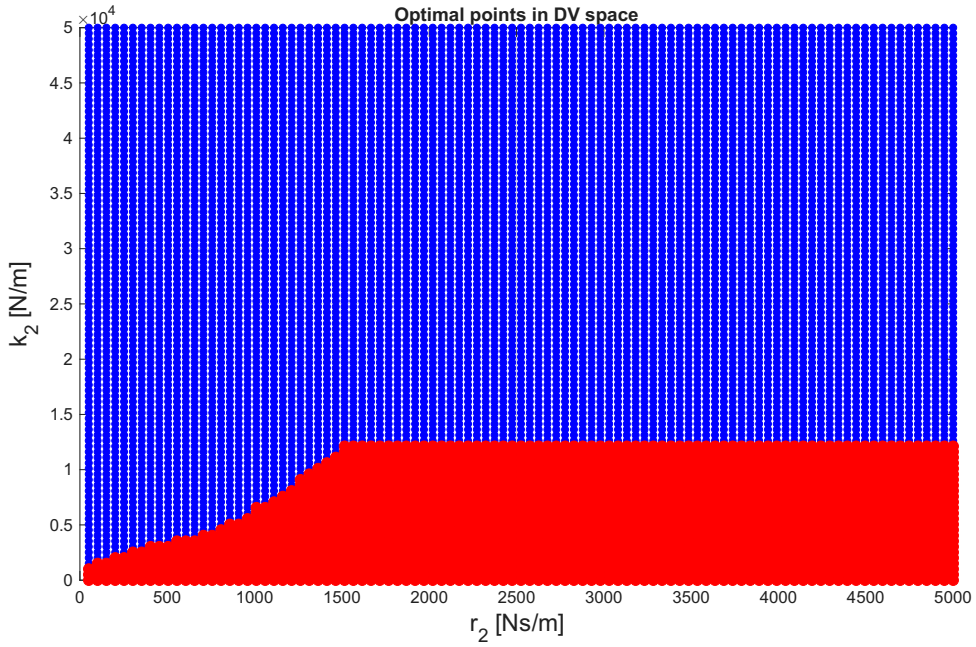


Figure 8: Pareto-optimal set in the space of the design variables (Case 2)

3 Lab 3: Passive suspension optimization (weighted sum, constraint method and analytical derivation of PO set)

3.1 Problem Definition

The aim of this laboratory is to solve the optimization problem presented in the first case of Lab 2 by using different approaches. In particular, the requirement is to apply scalarization methods, such as the weighted sum and the constraint method, and to compute the Pareto-optimal set through analytical derivation.

3.2 Matlab implementation

3.2.1 Weighted sum

The weighted sum method is a scalarization technique that treats a multi-objective optimization problem as a single-objective one. This approach requires the definition of an aggregate function $\phi(x)$, where $x = (k_2, r_2)$, representing the weighted sum of the two objective functions, $\sigma_{\ddot{x}_2}$ and σ_{F_z}, which are to be minimized. Each objective function is normalized according to its maximum value, ensuring that the optimization process is driven by the weights rather than the arbitrary numerical scales or units of the individual objectives. In this way, the objective functions become dimensionless quantities. The weights λ_1 and λ_2 vary between 0 and 1 and are chosen such that their sum is always equal to 1.

$$\min \phi(k_2, r_2) = \lambda_1 \frac{\sigma_{\ddot{x}_2}(k_2, r_2)}{\max(\sigma_{\ddot{x}_2})} + \lambda_2 \frac{\sigma_{F_z}(k_2, r_2)}{\max(\sigma_{F_z})} \tag{10}$$

$$0 \leq \lambda_1 \leq 1, \quad 0 \leq \lambda_2 \leq 1, \quad \lambda_1 + \lambda_2 = 1$$

The aggregate function $\phi(x)$ was implemented through a custom Matlab function `fun`, which takes as inputs the design variables contained in `x`, the weights in `lambda_used`, the other problem parameters, and the maximum values of the objective functions.

```
function f = fun(x,lambda_used,A,m1,m2,k1,max_discomfort,
    max_road_holding)
if x(1) < 0 || x(1) > 50000 || x(2) < 1 || x(2) > 5000
    f = 1e6;
    return
end
sigma_x2pp_fun = A * sqrt( ((m1 + m2) / (m2^2 * x(2))) * x(1)^2 ...
        + (k1 * x(2) / m2^2) ); % discomfort
sigma_Fz_fun = A * sqrt( ((m1 + m2)^3 / (m2^2 * x(2))) * x(1)^2 ...
        - 2 * (m1 * k1 * (m1 + m2) / (m2 * x(2))) * x(1) ...
        + (k1 * x(2) * (m1 + m2)^2 / m2^2) + (k1^2 * m1 / x(2)) ); ...
        % road holding
f = lambda_used(1) * sigma_x2pp_fun / max_discomfort + lambda_used(2) *
    ...
    sigma_Fz_fun / max_road_holding; % aggregate function
end
```

As can be seen, a penalty is added if the design variables fall outside their field of variation. Then, the values of k_2 and r_2 that minimize $\phi(k_2, r_2)$ are found by applying the `fminsearch` function inside a loop in which the weights λ_1 and λ_2 are varied. All the Pareto-optimal points in the space of design variables are stored in the vector `x_pareto`. For each optimal point, the relative values of the objective functions are then calculated.

```
lambda1 = linspace(0,1,500);
lambda2 = 1.-lambda1;
max_discomfort = max(ofs(:,1)); % maximum discomfort value taken from
    the vector of objevtive function  values ofs
max_road_holding = max(ofs(:,2)); % maximum road holding value taken
    from the vector of objevtive function  values ofs
x_pareto = zeros(length(lambda1),2);
for ii = 1:length(lambda1)
    lambda_used = [lambda1(ii) lambda2(ii)];
    x0 = [5000 2500]; % initial design point
    x = fminsearch(@(x) fun(x,lambda_used,A,m1,m2,k1,...
        max_discomfort,max_road_holding),x0); % Pareto-optimal design
    variables
    x_pareto(ii,1) = x(1);
    x_pareto(ii,2) = x(2);
end
sigma_x2pp_pareto = zeros(length(lambda1),1);
sigma_Fz_pareto = sigma_x2pp_pareto;
for ii=1:size(x_pareto,1)
    sigma_x2pp_pareto(ii) = A * sqrt( ((m1 + m2) / (m2^2 * x_pareto(ii
    ,2))) * x_pareto(ii,1)^2 ...
        + (k1 * x_pareto(ii,2) / m2^2) ); % Pareto-optimal solutions for
     discomfort
    sigma_Fz_pareto(ii) = A * sqrt( ((m1 + m2)^3 / (m2^2 * x_pareto(ii
    ,2))) * x_pareto(ii,1)^2 ...
        - 2 * (m1 * k1 * (m1 + m2) / (m2 * x_pareto(ii,2))) * x_pareto(
    ii,1) ...
        + (k1 * x_pareto(ii,2) * (m1 + m2)^2 / m2^2) + (k1^2 * m1 /
    x_pareto(ii,2)) ); ...
        % road holding
end
```

The results are shown in the pictures below.

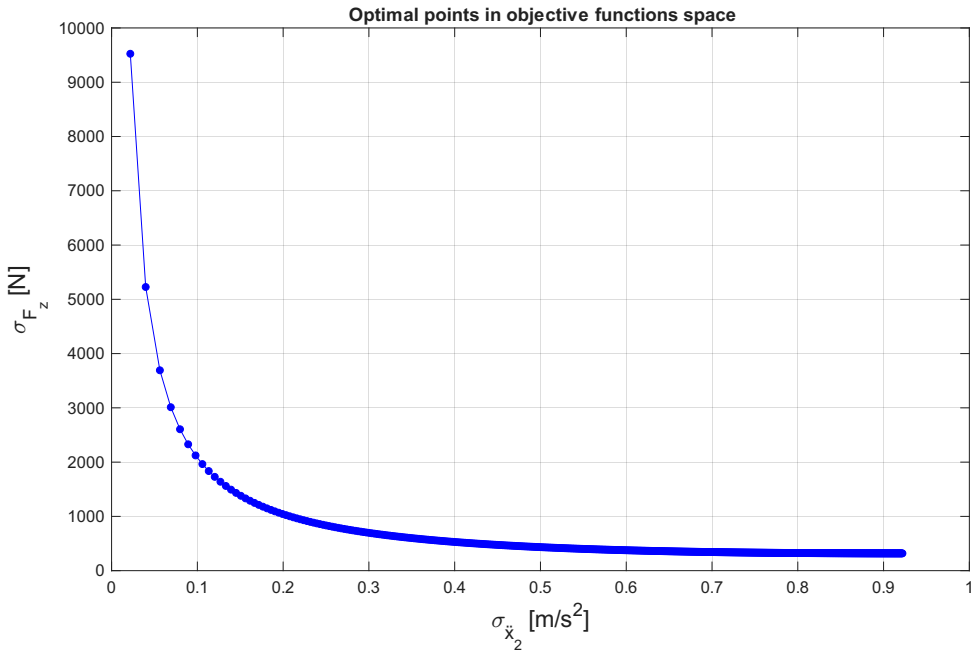


Figure 9: Pareto-optimal set in the space of the objective functions

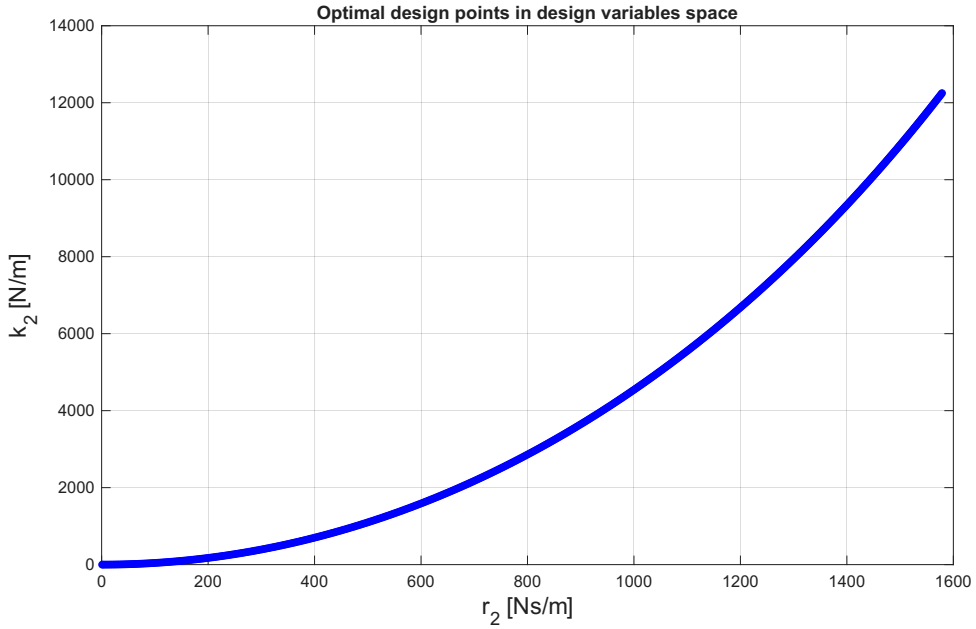


Figure 10: Pareto-optimal set in the space of the design variables

3.2.2 Constraint method

The constraint method is another scalarization technique, often more effective than the weighted sum because it can be used even for non-convex problems. It consists of minimizing a single objective function while treating the others as design constraints. In this case, $\sigma_{\ddot{x}_2}$ is the function to be minimized, while σ_{F_z} is treated as a design constraint.

$$\min \sigma_{\ddot{x}_2}(k_2, r_2)$$

$$\sigma_{F_z} \leq \epsilon_2$$

By varying ϵ_2, it is possible to compute different Pareto-optimal solutions. To obtain the entire Pareto-optimal set, ϵ_2 is varied between the minimum and the maximum values of σ_{F_z}.

The constraint method was implemented using the Matlab function `fmincon`, placed inside a loop where ϵ_2 varies across its field of variation. `fmincon` takes as input the custom function `sigma_x2pp_cm` that calculates the discomfort values for the design point defined by the array `x`. The optimization is performed starting from an initial design point `x0` and evaluated considering the design variable bounds and the active nonlinear constraint represented by the custom function `constr`.

The nonlinear constraint is defined according to the following equation:

$$\sigma_{F_z} - \epsilon_2 \leq 0 \tag{11}$$

Furthermore, `fmincon` utilizes an optimization algorithm specified in the `options` variable. The algorithms used to solve the problem are the SQP and the Interior-point algorithm. The desired algorithm can be selected via a dialog box before performing the minimization.

```
question = menu('Choose algorithm','SQP','Interior-point');
switch question
    case 1 % SQP method
options = optimoptions('fmincon','Algorithm','sqp',...
    'Display','off',...
    'StepTolerance',1e-10,...
    'OptimalityTolerance',1e-10);
    case 2 % Interior-point method
        options = optimset('Algorithm','interior-point');
end
% Solving algorithm for constraint method
epsilon_road_holding = linspace(min(ofs(:,2)),max(ofs(:,2)),1000); ...
    % Constraint values for road holding [N]
x_pareto2 = zeros(length(epsilon_road_holding),2);
for ii = 1 : length(epsilon_road_holding)
    eps_rh = epsilon_road_holding(ii);
    x0 = [10 10]; % initial design point
    x = fmincon(@(x) sigma_x2pp_cm(x, A, m2 , m1, k1),x0,[],[],[],[],...
        [0 0],[50000 5000],@(x) constr(x, A, m2 , m1, k1, eps_rh),...
        options); % Constraint method
    x_pareto2(ii,:) = x; % Pareto-optimal set of variables
end
```

The customized functions are showed below.

```
function f = sigma_x2pp_cm(x, A, m2 , m1, k1)
f = A * sqrt( ((m1 + m2) / (m2^2 * x(2))) * x(1)^2 ...
        + (k1 * x(2) / m2^2) ); % discomfort
end
```

```
function [c,ceq] = constr(x, A, m2 , m1, k1, eps_rh)
c = A * sqrt( ((m1 + m2)^3 / (m2^2 * x(2))) * x(1)^2 ...
        - 2 * (m1 * k1 * (m1 + m2) / (m2 * x(2))) * x(1) ...
        + (k1 * x(2) * (m1 + m2)^2 / m2^2) + (k1^2 * m1 / x(2)) ) -
    eps_rh; % nonlinear constraint
ceq = []; % linear constraint
end
```

The pictures below show the results given by each algorithm. The Pareto-optimal points in the space of the objective functions were found calculating the values of the objective functions for each optimal design point.

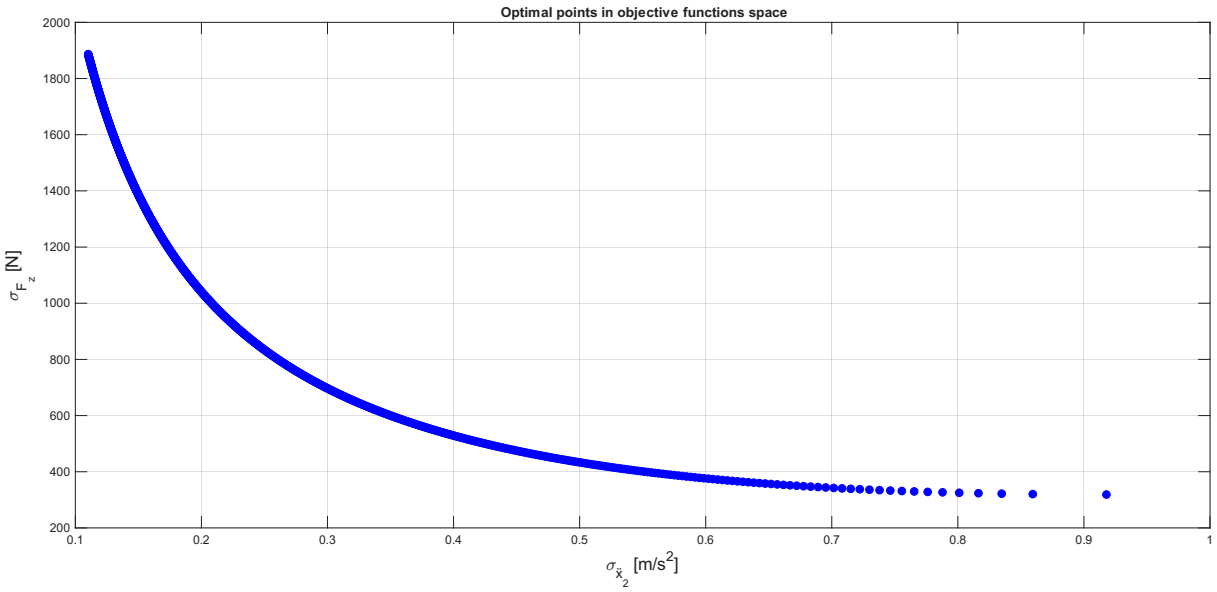


Figure 11: Pareto-optimal set in the space of the objective functions (SQP)

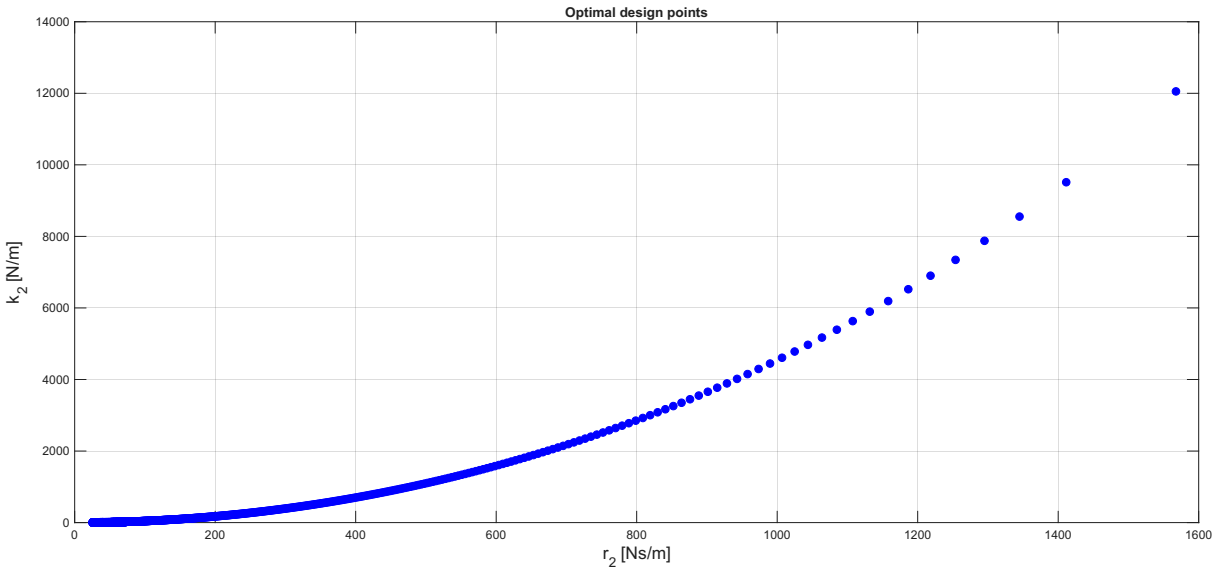


Figure 12: Pareto-optimal set in the space of the design variables (SQP)

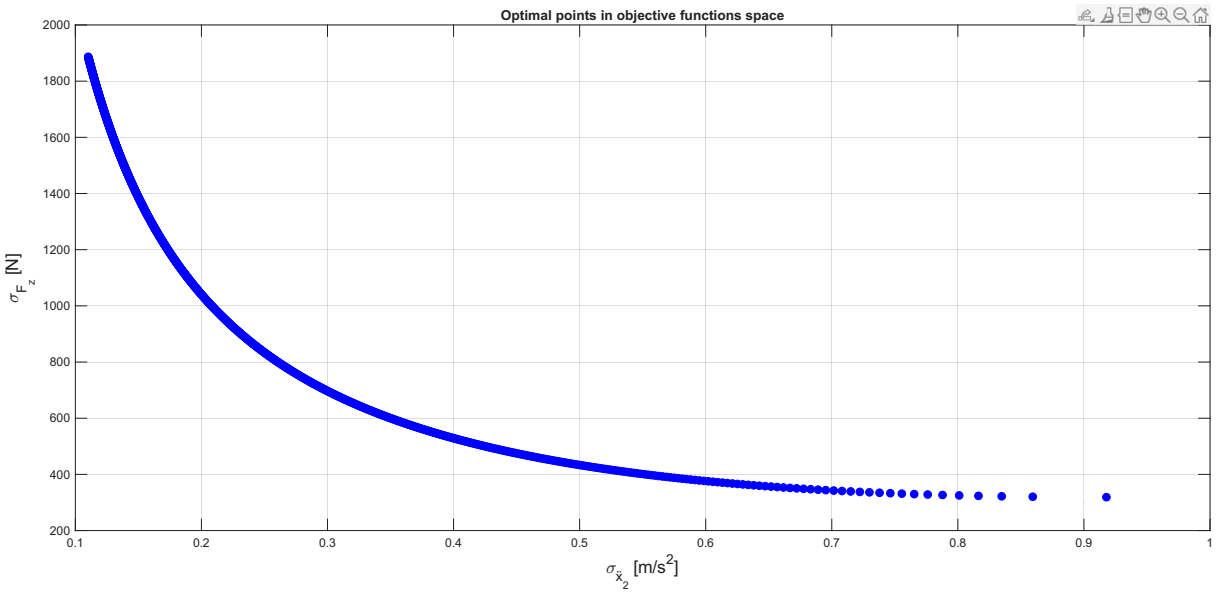


Figure 13: Pareto-optimal set in the space of the objective functions (Interior-point)

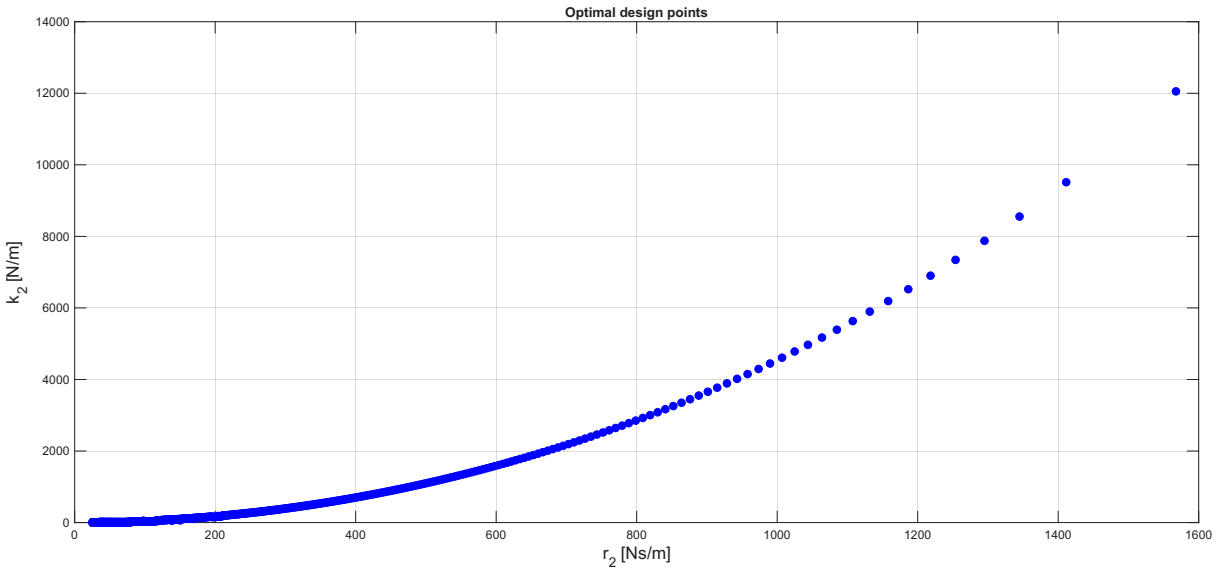


Figure 14: Pareto-optimal set in the space of the design variables (Interior-point)

3.2.3 Analytical derivation of the Pareto-optimal set

The analytical derivation of the Pareto-optimal set is based on the Fritz-John necessary condition for unconstrained problems with two objective functions.

In this particular case, the Fritz-John condition is represented by the following equation:

$$\frac{\partial \sigma_{\ddot{x}_2}}{\partial k_2} \cdot \frac{\partial \sigma_{F_z}}{\partial r_2} = \frac{\partial \sigma_{\ddot{x}_2}}{\partial r_2} \cdot \frac{\partial \sigma_{F_z}}{\partial k_2} \tag{12}$$

However, not all the points that satisfy this condition are Pareto-optimal. For a point to be Pareto-optimal, the scalar product of the gradients of the objective functions evaluated at that point must be negative or equal to zero:

$$\nabla \sigma_{\ddot{x}_2}^T \nabla \sigma_{F_z} \leq 0 \tag{13}$$

Thus, the Pareto front is bounded by the individual minima of each objective function.

The problem was solved by treating the design variables as symbolic variables to compute the Fritz-John equation. Solving this equation provides an analytical relation $r_2 = f(k_2)$ that contains the Pareto-optimal set in the design variables domain. Subsequently, the variables were parametrized using a symbolic parameter `t`, which was then substituted with discrete values within the range of k_2. The Pareto-optimal solutions were determined by evaluating the scalar product between the gradients of the objective functions. Points yielding a negative product are considered Pareto-optimal.

```
syms r2 k2 positive
Dsymb = A*sqrt( ((m1+m2)*k2^2) / (m2^2*r2) + (k1*r2)/m2^2); % discomfort
RHsymb = A * sqrt( ((m1 + m2)^3 * k2^2) / (m2^2 * r2) ...
        - 2 * (m1 * k1 * (m1 + m2) * k2) / (m2 * r2) ...
        + (k1 * r2 * (m1 + m2)^2) / (m2^2) ...
        + (k1^2 * m1) / r2); % road holding
Dr2 = diff(Dsymb, r2);
Dk2 = diff(Dsymb, k2);
RHr2 = diff(RHsymb, r2);
RHk2 = diff(RHsymb, k2);
Jacobian = Dk2*RHr2 - Dr2*RHk2; % Fritz-John condition
Jacobian_simplified = simplify(Jacobian);
k2_solved = solve(Jacobian_simplified, r2); % r2 = f(k2)
syms t positive
k2_t = t; % (k2 parametrized)
r2_t = subs(k2_solved(1,1), k2, t); % (r2 parametrized) = f(t)
t_vals = linspace(0,50000,100000);
k2_fun = matlabFunction(k2_t, 'Vars', t);
r2_fun = matlabFunction(r2_t, 'Vars', t);
k2_pareto = k2_fun(t_vals);
r2_pareto = r2_fun(t_vals);

Dpar = zeros(1,size(k2_pareto,2));
RHpar = Dpar;
for ii = 1:size(k2_pareto,2)
    Dpar(ii) = A*sqrt( ((m1+m2)*k2_pareto(ii)^2) / (m2^2*r2_pareto(ii)) + (k1*r2_pareto(ii))/m2^2);
    RHpar(ii) = A * sqrt( ((m1 + m2)^3 * k2_pareto(ii)^2) / (m2^2 * r2_pareto(ii)) ...
        - 2 * (m1 * k1 * (m1 + m2) * k2_pareto(ii)) / (m2 * r2_pareto(ii)) ...
        + (k1 * r2_pareto(ii) * (m1 + m2)^2) / (m2^2) ...
        + (k1^2 * m1) / r2_pareto(ii));
end

Dr2_t = subs(Dr2,[k2 r2],[t r2_t]);
Dk2_t = subs(Dk2,[k2 r2],[t r2_t]);
RHr2_t = subs(RHr2,[k2 r2],[t r2_t]);
RHk2_t = subs(RHk2,[k2 r2],[t r2_t]);
Dr2_fun = matlabFunction(Dr2_t, 'Vars', t);
Dk2_fun = matlabFunction(Dk2_t, 'Vars', t);
RHr2_fun= matlabFunction(RHr2_t, 'Vars', t);
RHk2_fun = matlabFunction(RHk2_t, 'Vars', t);
Dr2_val = Dr2_fun(t_vals);
Dk2_val = Dk2_fun(t_vals);
RHr2_val = RHr2_fun(t_vals);
RHk2_val = RHk2_fun(t_vals);
```

```
grad_product = (Dr2_val .* RHr2_val) + (Dk2_val .* RHk2_val); % vector
    of scalar products between gradients
index = grad_product <= 0;
r2_par = r2_pareto(index); % Pareto-optimal r2
k2_par = k2_pareto(index); % Pareto-optimal k2
Dpareto = Dpar(index); % Pareto-optimal discomfort
RHpareto = RHpar(index); % Pareto-optimal road holding
```

The final results are shown below.

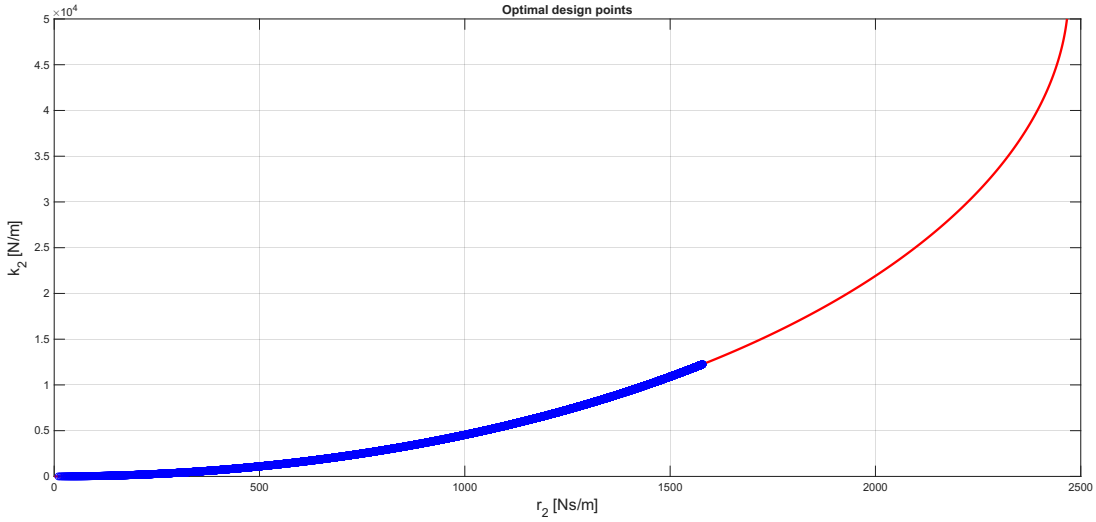


Figure 15: Pareto-optimal set in the space of the design variables

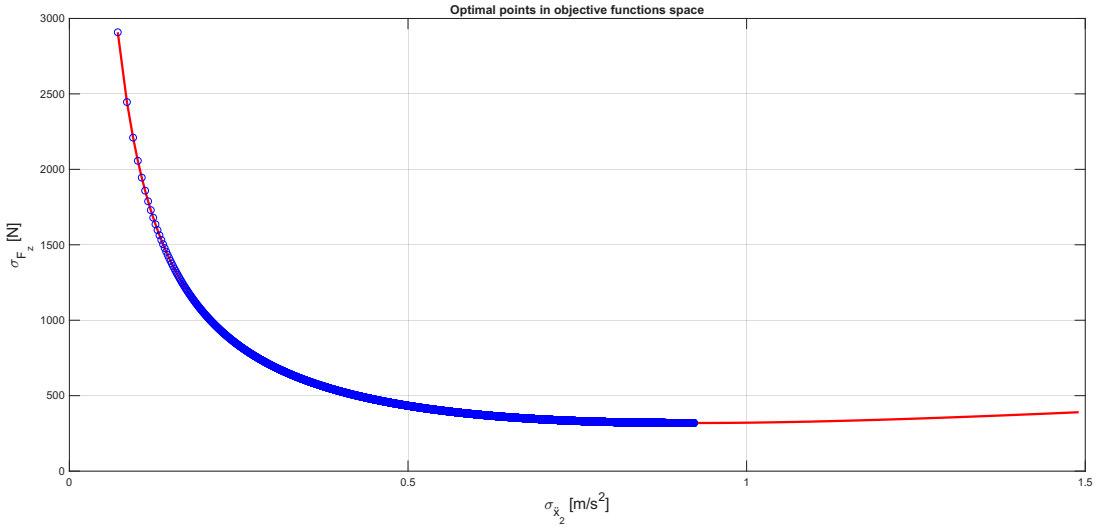


Figure 16: Pareto-optimal set in the space of the objective functions

As can be seen, the Pareto-optimal solutions are those highlighted in blue.

4 Lab 4: Active suspension optimization

4.1 Problem Definition

The aim of this laboratory is to optimize the active suspension system of a vehicle based on a 2-DOF (Degrees of Freedom) quarter-car model.

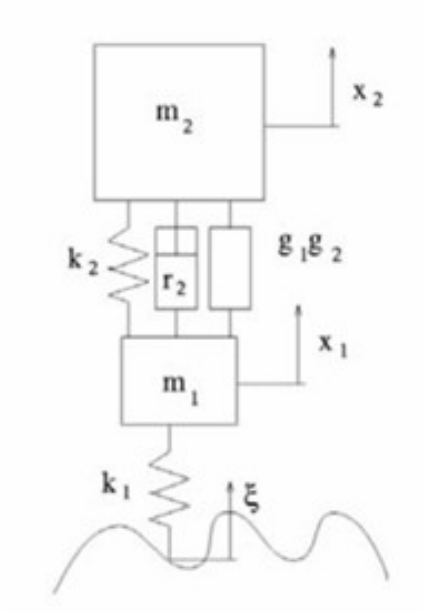


Figure 17: Active suspension quarter-car vehicle model

The parameters are defined as follows:

- m_1: unsprung mass [kg];
- m_2: sprung mass [kg];
- k_1: tyre radial stiffness [N/m];
- k_2: suspension equivalent stiffness (assumed linear) [N/m];
- r_2: suspension equivalent damping (assumed linear) [Ns/m];
- g_1: actuator gain (stiffness) [N/m];
- g_2: actuator gain (sky-hook damping) [N/m];
- ξ_1: road irregularity [m];

The primary difference compared to a passive suspension is the presence of an actuator that introduces energy into the system. Specifically, the actuator opposes the vertical movement of the cabin as the vehicle interacts with road irregularities. The operating principle of the actuator is based on the "sky-hook" configuration, which conceptually "hangs" the cabin to a fixed point in the sky via a virtual spring and damper. Using this configuration, the force exerted by the actuator is calculated as:

$$F_{actuator} = g_1(x_2 - x_1) + g_2\dot{x_2} \tag{14}$$

- $F_{actuator}$: force exerted by the actuator [N];
- $x_2 - x_1$: relative displacement between the wheel and the cabin [m];
- $\dot{x_2}$: vertical absolute velocity of the vehicle body $[\frac{m}{s}]$.

The actuator's activity influences both passenger comfort and road holding, which are expressed using different formulas than those used for passive suspensions:

$$\sigma_{\ddot{x}_2} = A \cdot \sqrt{\frac{k_1 g_1}{m_2 g_2}} \tag{15}$$

$$\sigma_{F_z} = A \cdot \sqrt{\frac{g_1 k_1 (m_1 + m_2)^2}{g_2 m_2} - \frac{k_1^2 (2m_1 + m_2)}{g_2} + \frac{k_1^3 m_2}{g_1 g_2} + \frac{g_2 k_1^2 m_1}{g_1 m_2}} \tag{16}$$

$$A = \sqrt{\frac{A_b v}{2}} \tag{17}$$

As in previous labs, the objective is to minimize discomfort ($\sigma_{\ddot{x}_2}$) and tire load fluctuations (σ_{F_z}). In this study, the design variables are g_1 and g_2, while k_2 and r_2 are assumed to be zero.

Reference values for the parameters are reported in Table 3.

Parameter	**Value**
Road roughness coefficient A_b	1.4e-5 m
Velocity v	30 m/s
Sprung mass m_2	230 kg
Unsprung mass m_1	30 kg
Tyre radial stiffness k_1	120,000 N/m

Table 3: Problem parameters

The design variable g_1 varies between 0 N/m and 400,000 N/m, while g_2 varies between 0 Ns/m and 100,000 Ns/m.

4.2 Matlab implementation

The problem was solved by applying the constraint method using the interior-point optimization algorithm. The Matlab code follows the same structure used in Lab 3, with the objective function formulas updated for the active suspension case.

The results are shown below.

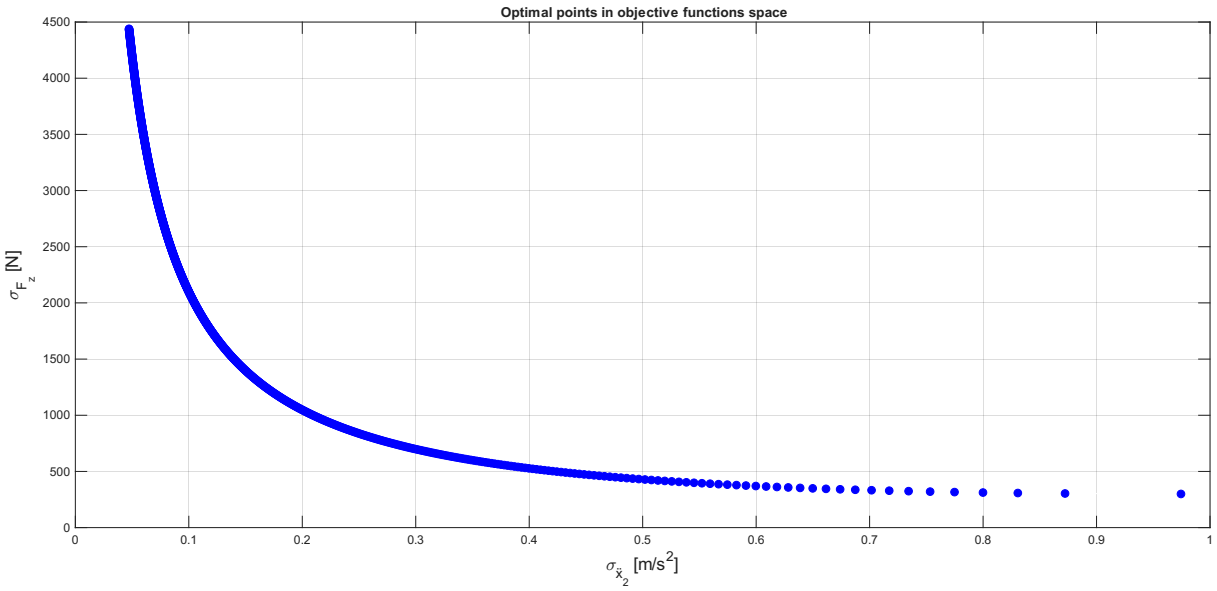


Figure 18: Pareto-optimal set in the space of the objective functions

4.3 Comparison with passive suspension results

The Pareto-optimal set of the active suspension was compared with that of the passive suspension.

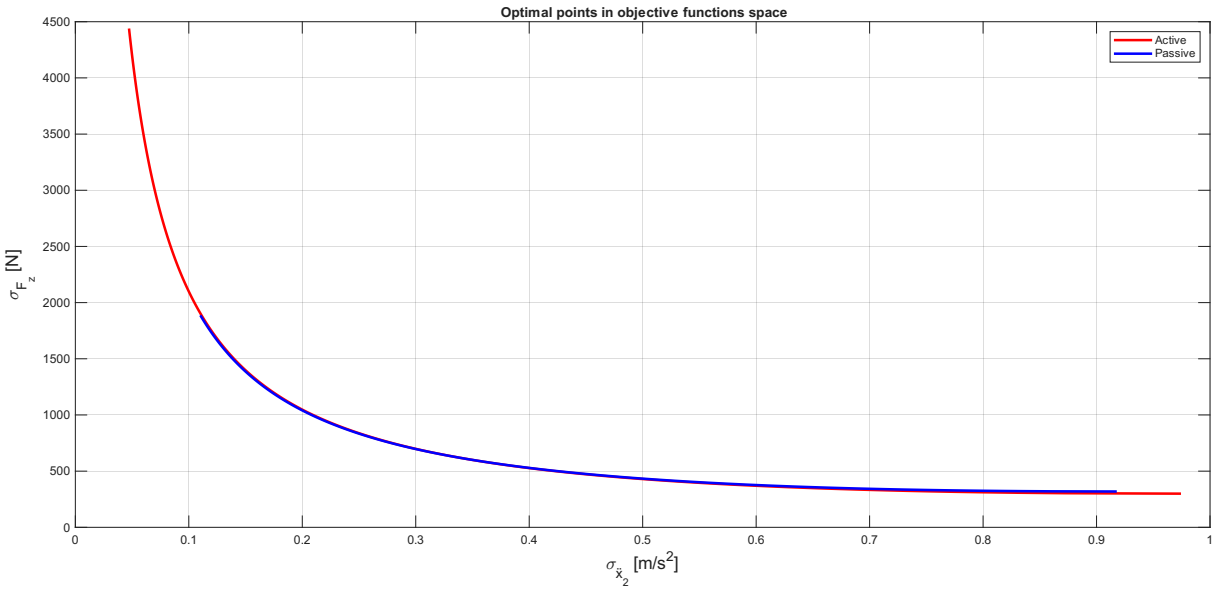


Figure 19: Active suspension vs. passive suspension comparison

As shown in the graph, the active suspension can achieve significantly lower discomfort values compared to the passive system. However, reaching these extreme comfort levels results in a substantial increase in tire load fluctuation, thereby degrading road holding. At comparable levels of discomfort, there is no significant difference in road holding performance. Furthermore, at higher discomfort values, the active suspension actually exhibits lower road holding values compared to the passive one.

5 Lab 6: Genetic Algorithms (Optimization of a Paraboloid)

5.1 Problem Definition

The aim of this laboratory is to implement a Genetic Algorithm (GA) in Matlab to find the maximum value of the function:

$$y(x_1, x_2) = 100 - (x_1^2 + x_2^2) \tag{18}$$

subject to the domain constraints $x_1, x_2 \in [0, 5]$.

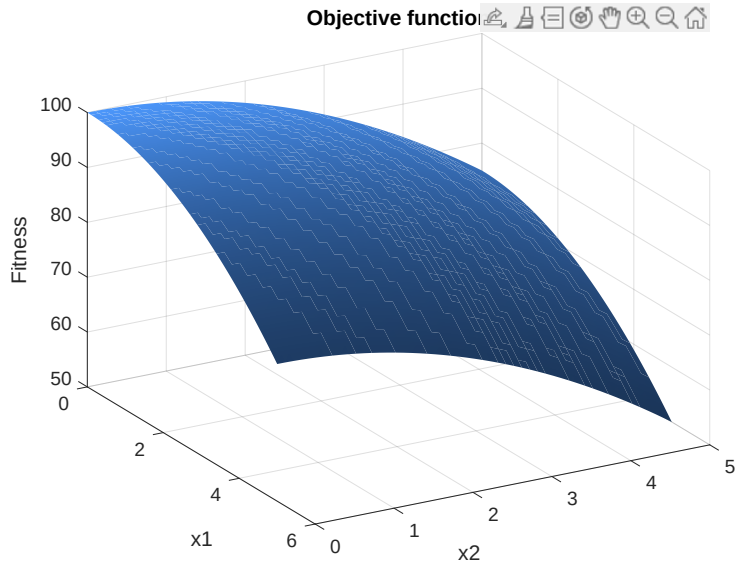


Figure 20:graphical representation of the objective function

5.2 Matlab Implementation

The implemented Genetic Algorithm is governed by seven key parameters. The simulation operates on a population size of 100 individuals evolving over a maximum of 200 generations. Regarding the encoding, the chromosome length was set to 16 bits; this parameter is crucial as it defines the discretization resolution of the design variable space, which is bounded by the domain constraints $x_1, x_2 \in [0, 5]$. The evolutionary process is driven by a crossover probability of 100%, representing the total involvement of parents in reproduction, and a mutation probability essential for maintaining genetic diversity. Finally, the selection strategy selects the fittest individuals from the combined pool of parents and offspring. To analyze the impact of different genetic operators the algorithm was tested in three different configurations, but starting from the same initial population to ensure the *consistency* of the comparison of the results

```
seed = 42;
rng(seed)
x_inital = 5*rand(100,2);
```

To evaluate the convergence speed and the quality of the optimization process, the **Average Fitness** of the population was monitored at each generation. This metric was chosen as it provides a clear indication of the global improvement of the genetic pool over time.

5.2.1 Case 1: Standard GA

For the first simulation the GA was set in the following way:

Parameter	Value
Population Size	100 individuals
Chromosome Length	16 bits
Domain Constraints	$x_1, x_2 \in [0, 5]$
Crossover Probability	100%
Mutation Probability	-
Selection Strategy	No selection
Max Generations	200

Table 4: Case 1 GA Setup Parameters.

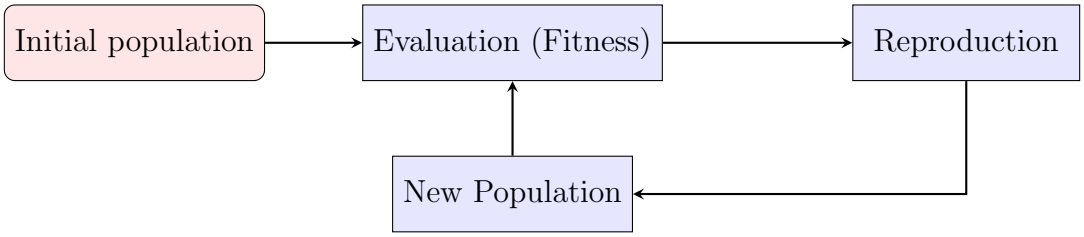


Figure 21:Flowchart of the standard Genetic Algorithm.

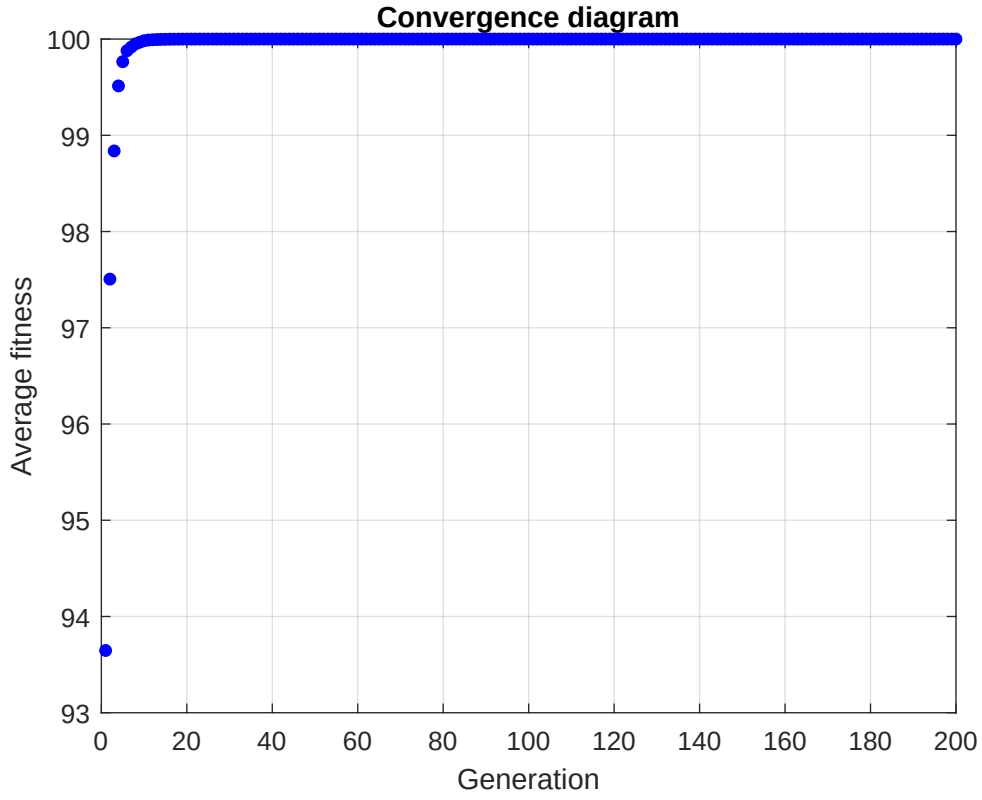


Figure 22

The results for Case 1 (Fig. 22) demonstrate a significant limitation of using *crossover* alone. The algorithm exhibits slow convergence and eventually stagnates before reaching the global optimum ($y = 100$).

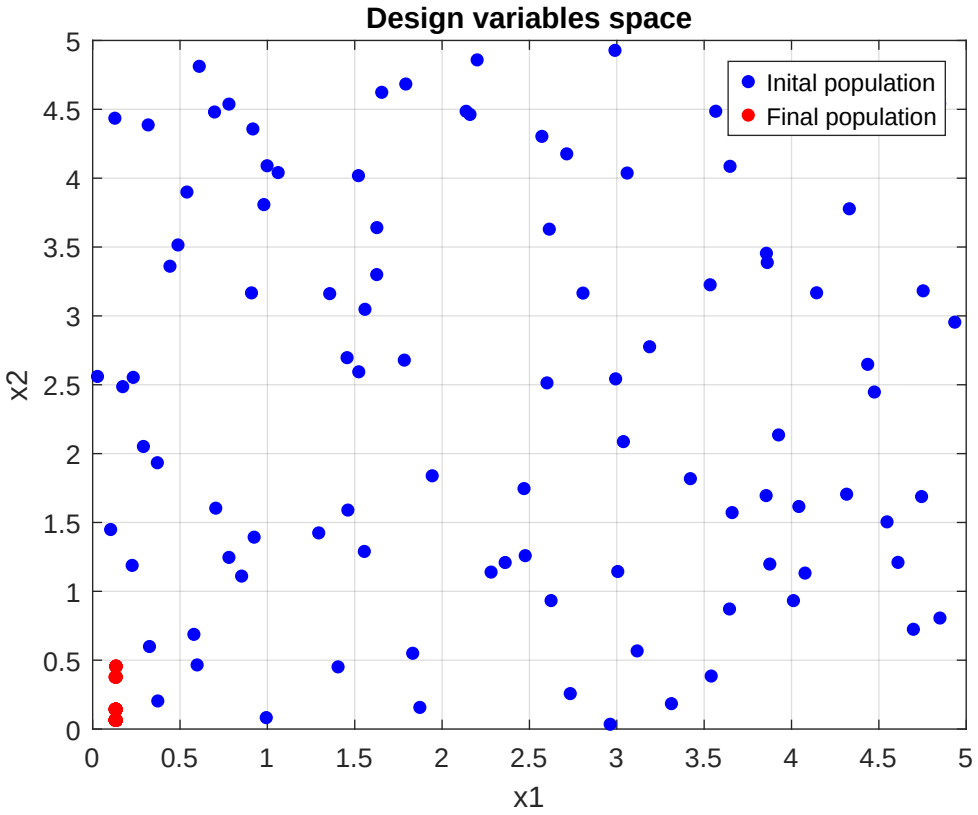


Figure 23

This stagnation occurs because the algorithm is strictly limited by the genetic information contained in the initial population. If a specific gene (bit value) required to construct the optimal solution is missing from the initial pool, the crossover operator—which only recombines existing parts—cannot generate it. Consequently, the algorithm depends entirely on the "luck" of the initialization phase. To overcome this "loss of diversity," a mutation operator is required.

5.2.2 Case 2: Standard GA:mutation added

The mutation operator ensures genetic diversity by introducing random bit flips. In the provided implementation, the mutation probability p is not fixed arbitrarily but is adapted to the population dimensions.

The algorithm calculates two reference thresholds based on the population size ($N_{ind} = 100$) and the chromosome length ($N_{genes} = 16$):

- A lower bound proportional to the population size: $1/N_{ind} = 0.01$.
- An upper bound proportional to the chromosome length: $1/N_{genes} = 0.0625$.

The operational probability p is then derived by linearly interpolating between these bounds using the control parameter `par`:

$$p = \text{par} \cdot p_{min} + (1 - \text{par}) \cdot p_{max} \tag{19}$$

where $p_{min} = \min(1/N_{ind}, 1/N_{genes})$ and $p_{max} = \max(1/N_{ind}, 1/N_{genes})$.

Once p is determined, a Boolean mask is generated using random uniform sampling:

$$M_{mut} = \text{rand}(N_{ind}, N_{genes}) < p \tag{20}$$

The genes identified by the mask M_{mut} are then inverted. In this experiment, setting `par` $= 0.5$ resulted in a balanced mutation probability (approx. 2%).

Parameter	**Value**
Population Size	100 individuals
Chromosome Length	16 bits
Domain Constraints	$x_1, x_2 \in [0, 5]$
Crossover Probability	100%
Mutation Probability	2% (par = 0.5)
Selection Strategy	No selection
Max Generations	200

Table 5: Case 2 GA Setup Parameters.

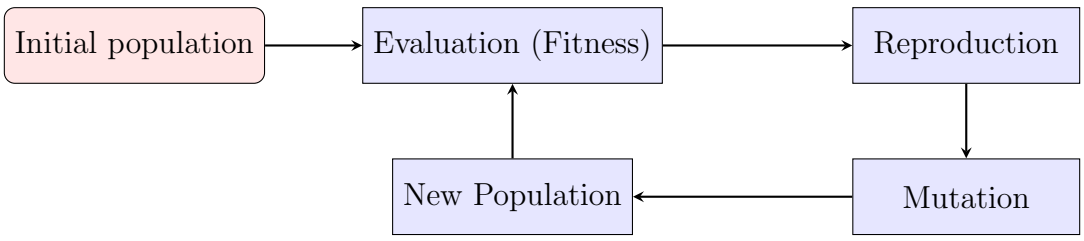


Figure 24:Flowchart of the SGA with mutation.

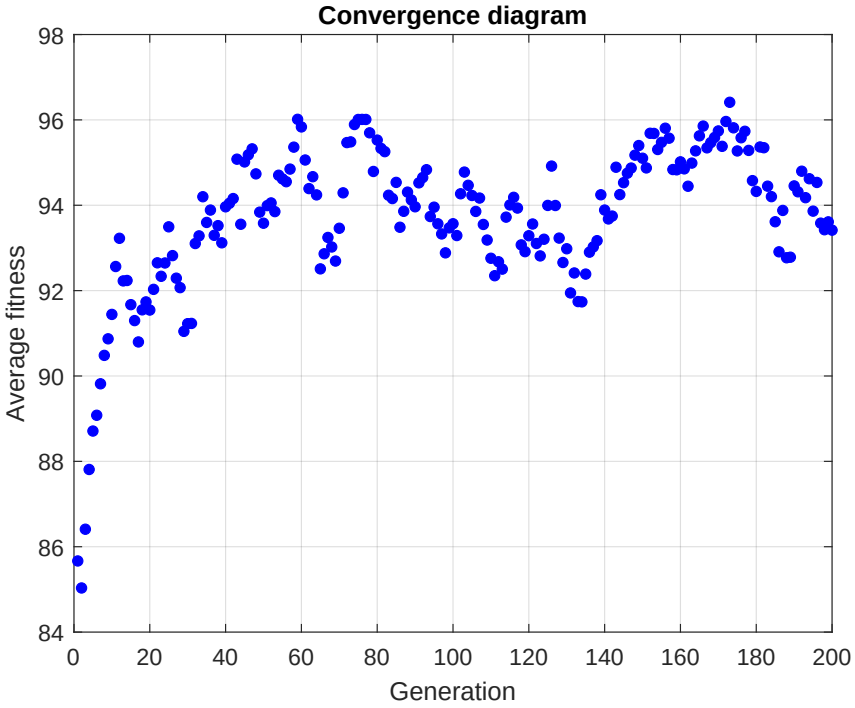


Figure 25

As shown in Figure 25, while mutation prevents stagnation, it introduces *instability*. The convergence curve is characterized by significant noise and fluctuations.

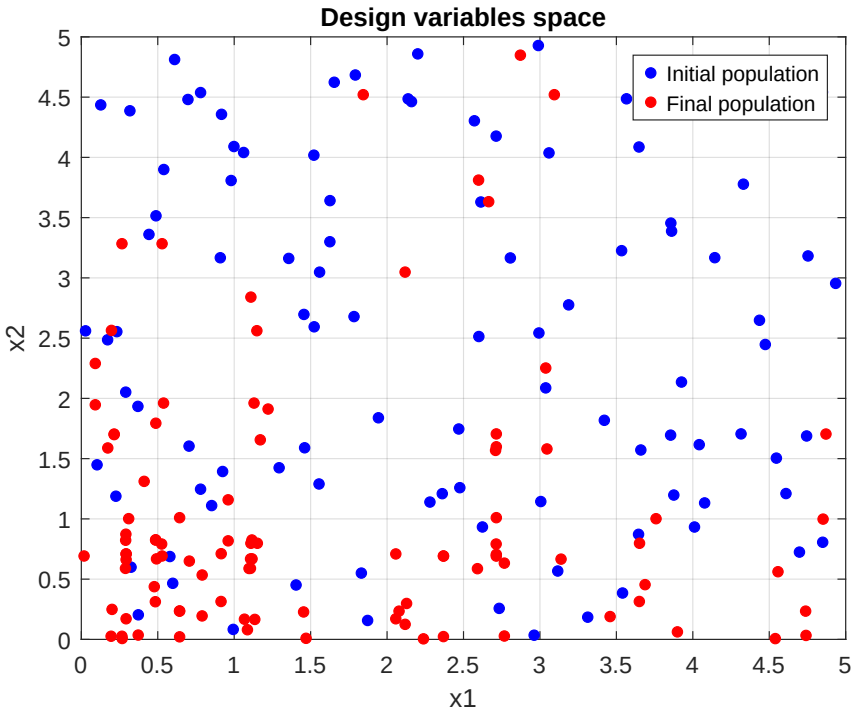


Figure 26

This behavior is due to the non-elitist nature of the algorithm used in this step. Since the algorithm unconditionally replaces parents with offspring (regardless of whether the offspring are better or worse), deleterious mutations are retained in the population, causing the average fitness to drop repeatedly.

5.2.3 Case 3: Standard GA:elitism strategy

To stabilize the convergence and ensure monotonic growth, a *Elitist Selection* strategy (implemented via the `ga_discard` function) was applied. This mechanism compares the fitness of both parents and offspring, selecting only the best 100 individuals to form the next generation.

Parameter	**Value**
Population Size	100 individuals
Chromosome Length	16 bits
Domain Constraints	$x_1, x_2 \in [0, 5]$
Crossover Probability	100%
Mutation Probability	2% (par = 0.5)
Selection Strategy	Elitism(discard)
Max Generations	200

Table 6: Case 3 GA Setup Parameters.

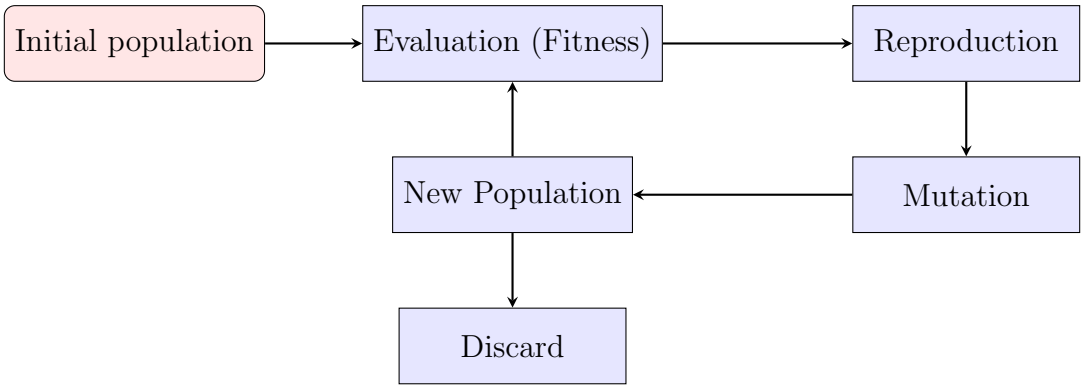


Figure 27:Flowchart of the SGA with mutation and discard.

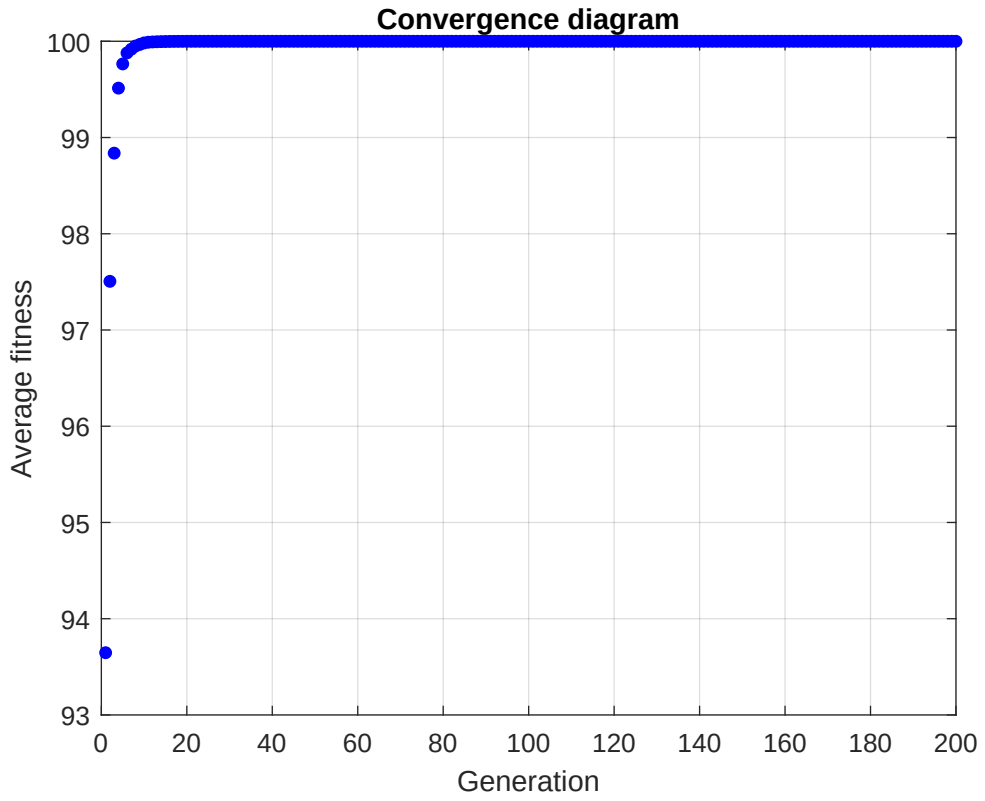


Figure 28

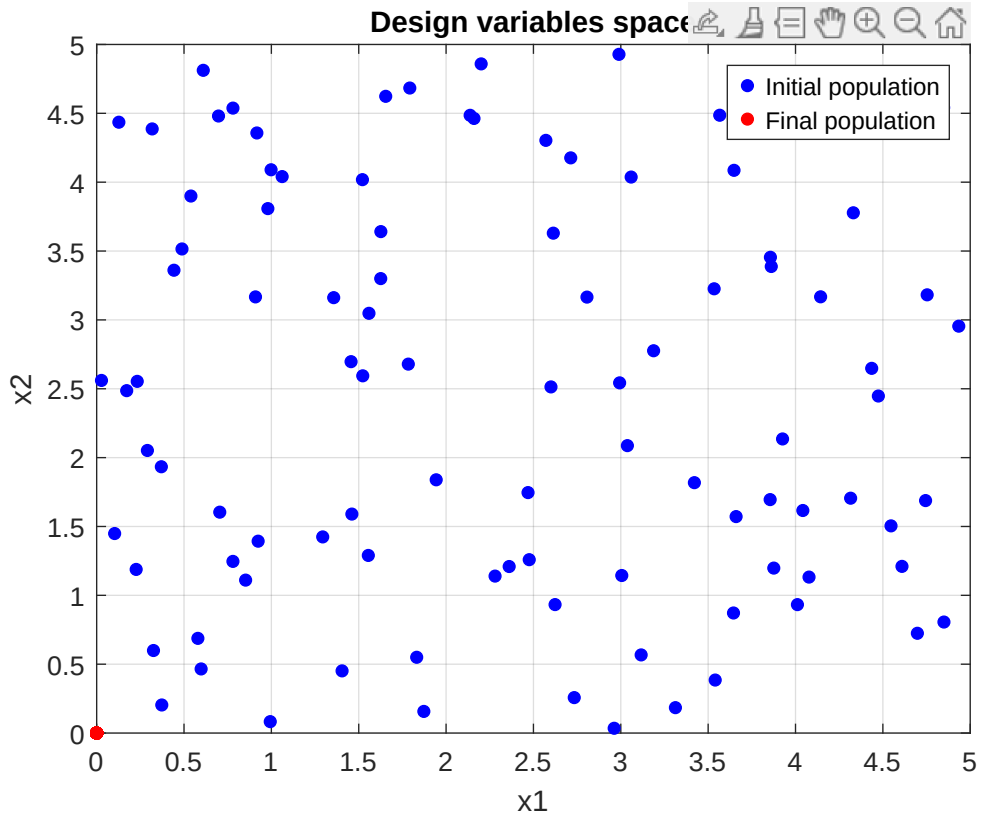


Figure 29

5.3 Comparing results

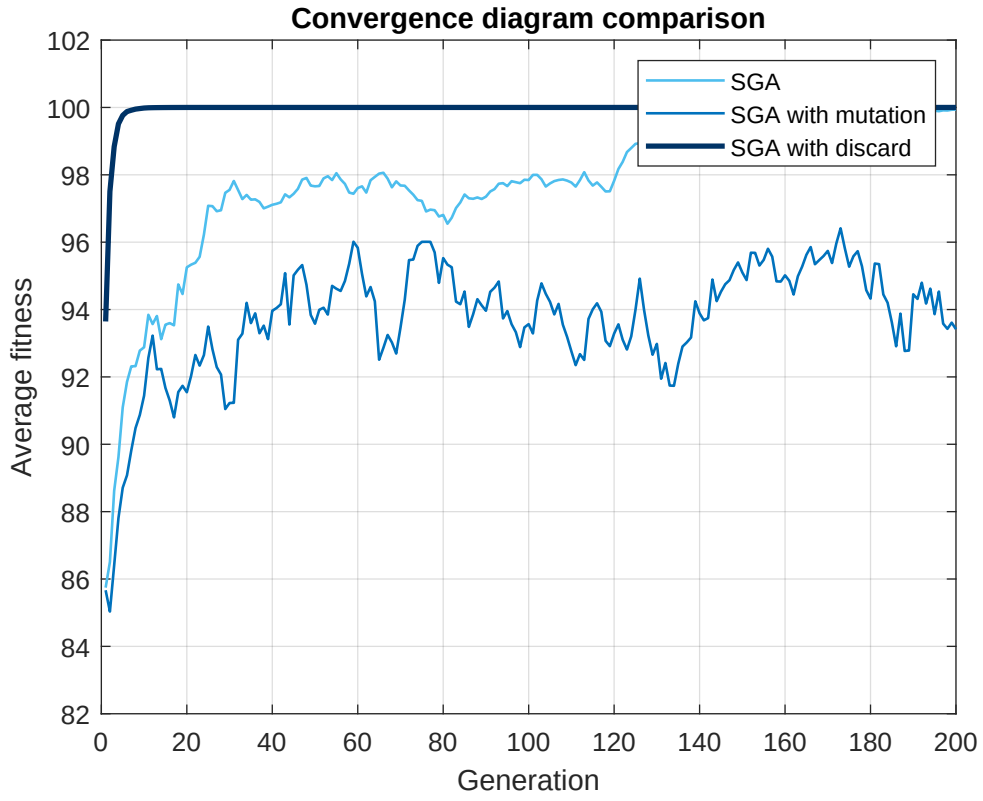


Figure 30

The comparative analysis (Fig. 41) clearly highlights the superiority of the Elitist approach (Case 3, dark blue line). Unlike Case 1, which stagnates, Case 3 reaches the global maximum thanks to the diversity provided by mutation. Unlike Case 2, which oscillates, Case 3 shows a smooth, *monotonic convergence*. The elitist strategy acts as a "filter," accepting the beneficial mutations that explore the search space while discarding the destructive ones, thus ensuring that the population's average fitness never decreases.

6 Lab 7: GA Multi-objective optimization

6.1 Double wishbone suspension design

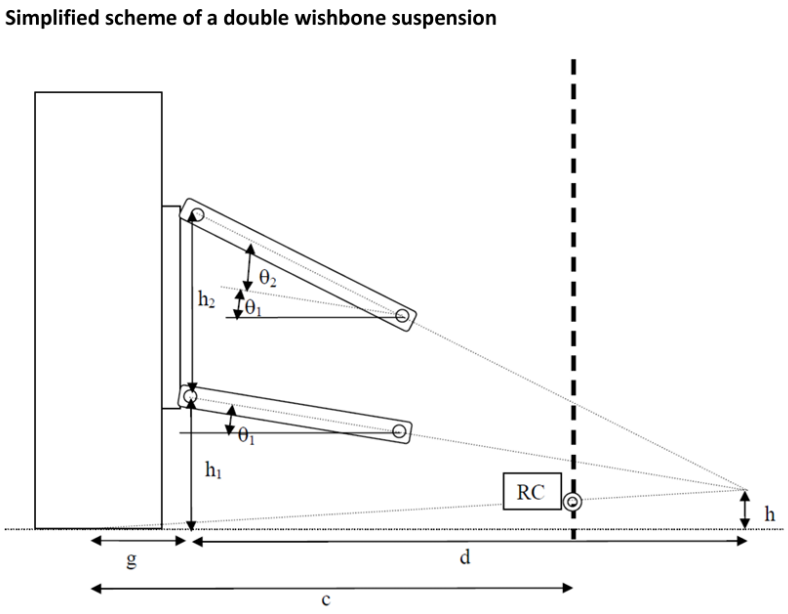


Figure 31

- The camber rate of change is given by:

$$\frac{\Delta\gamma}{\Delta z} = \frac{1}{d+g}$$

- The height of the roll center (RC) is:

$$RCH = \frac{h}{d+g}c$$

The Geometrical parameters d e h are not constant, but they are function of: θ_1 e θ_2:

$$d = d(\theta_1, \theta_2) \quad \text{e} \quad h = h(\theta_1, \theta_2)$$

Consequentially, θ_1 e θ_2 are the *Design Variables* (DV).

Variabile	**Lower Bound**	**Upper Bound**
θ_1	$-9[deg]$	$+1[deg]$
θ_2	$+5[deg]$	$+15[deg]$

Table 7: Design Variables boundaries.

Targets on the roll centre and on the gradient on camber angle are given

Parameter	Target value
$TRCH$	100 [mm]
$T\Delta\gamma\|_{\Delta z=50\text{ mm}}$	2 [deg]

So the two objective functions to minimize are the mean squared error of the the function and their respective targets.The starting setup of the GA was set

The first objective function, f_1, minimizes the normalized squared error between the calculated Roll Center Height (RCH) and the target value ($TRCH = 100$ mm):

$$f_1(\mathbf{x}) = \left(\frac{RCH(\mathbf{x}) - TRCH}{TRCH} \right)^2 \tag{21}$$

The second objective function, f_2, minimizes the normalized squared error of the camber variation at a vertical wheel displacement of $\Delta z = 50$ mm. The target gradient is $T\Delta\gamma = 2°$:

$$f_2(\mathbf{x}) = \left(\frac{\Delta\gamma(\mathbf{x})|_{\Delta z=50mm} - T\Delta\gamma}{T\Delta\gamma} \right)^2 \tag{22}$$

6.2 Optimization Methodology

To solve this multi-objective problem, a Genetic Algorithm (GA) was implemented. Unlike single-objective optimization, there is no single best solution, but rather a set of trade-off solutions known as the Pareto Optimal Set.

6.2.1 Fitness Assignment via Pareto Ranking

The selection process relies on a rank-based fitness assignment to handle the dominance relationships between individuals:

1. **Non-Dominated Sorting:** At each generation, the algorithm identifies non-dominated individuals (those that cannot be improved in one objective without worsening the other). These form the current Pareto front and are assigned **Rank 1**.

2. **Ranking Loop:** Rank 1 individuals are temporarily removed from the population. The algorithm scans the remaining subset to find the new non-dominated front, which is assigned **Rank 2**. This "peeling" process repeats until all individuals are ranked.

3. **Fitness Calculation:** To favor Pareto-optimal solutions during selection, fitness is assigned inversely proportional to the rank:

$$Fitness = \frac{1}{Rank} \tag{23}$$

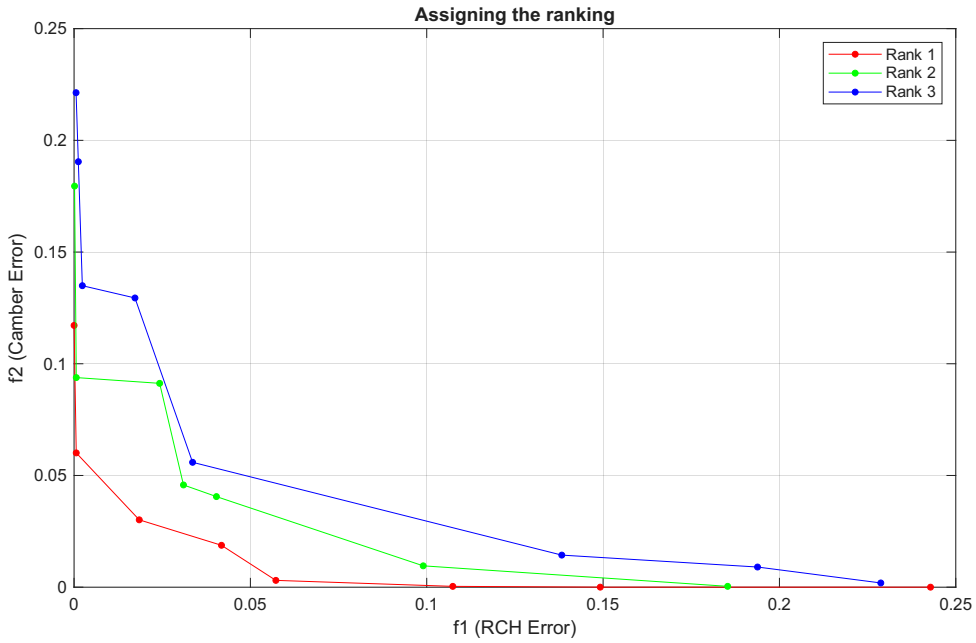


Figure 32Initial population in the objective space (f_1 vs f_2).

6.3 Simulation Results and Comparative Analysis

Two different configurations of the Genetic Algorithm were tested to evaluate convergence and population diversity.

Parameter	**Value**
Population Size	150 individuals
Chromosome Length	16 bits
Crossover Probability	100%
Mutation Probability	2% (par = 0.5)
Selection Strategy	Elitism(discard)
Max Generations	300

Table 8: GA Setup Parameters.

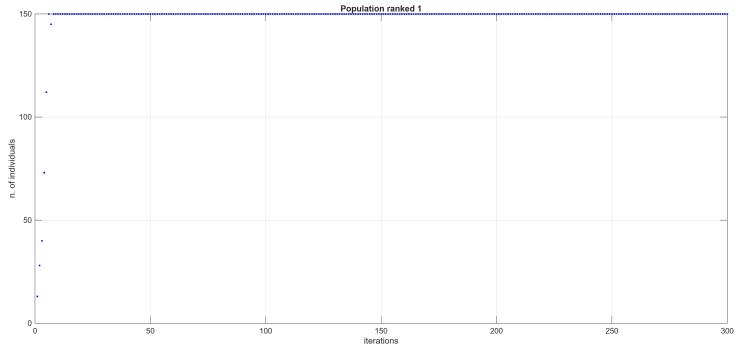


Figure 33: number of Pareto points

The immediate domination of Rank 1 individuals indicates a tendency towards premature convergence. The standard mutation operator appears unable to maintain sufficient population diversity to escape local optima. Attempts to rectify this by adjusting the mutation parameter *par* proved ineffective, as the final population remained largely unchanged.

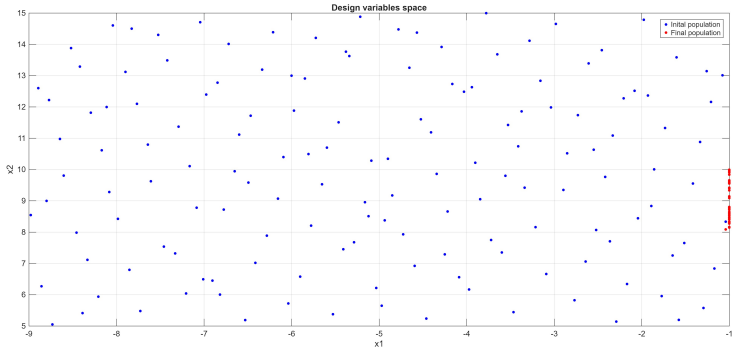


Figure 34:.

The analysis of the design variable space reveals that the solutions are heavily constrained by the upper bound of θ_1.

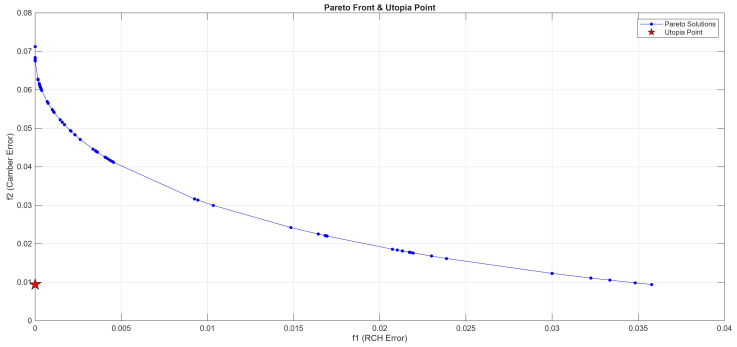


Figure 35Pareto front.

Trying to improve the diversity of the Pareto front a Sharing Function (Niche Count) was introduced. This technique modifies the fitness by penalizing individuals that are densely crowded in the objective space. The implementation of the Sharing Function relies on the calculation of the Euclidean distance between individuals in the objective space. During preliminary testing, it was observed that the two objective functions, f_1 and f_2, operated on different scales. To address this "out of scale" issue and ensure a balanced distribution, the objective values were dynamically normalized to the range $[0, 1]$ at each generation before computing the niche count.

```
%% --- 3. SHARING FUNCTION  ---
    if sharing_enabled
        sigma_share = 0.01;
        alpha = 2;

        objs = mat(:, 3:4);
        N = size(mat, 1);


```

```
        O_min = min(objs);
        O_max = max(objs);
        Range = O_max - O_min;
        Range(Range == 0) = 1;


        objs_norm = (objs - O_min) ./ Range;


        m = zeros(N, 1);


        for i = 1:N
            sh_sum = 0;
            for j = 1:N
               dist = sqrt(sum((objs_norm(i,:) - objs_norm(j,:)).^2));
                if dist < sigma_share
                    sh_func = 1 - (dist / sigma_share)^alpha;
                    sh_sum = sh_sum + sh_func;
                end
            end

            m(i) = sh_sum;
        end

        % Fitness = RawFitness / NicheCount
        mat(:,6) = mat(:,6) ./ m;
    end
```

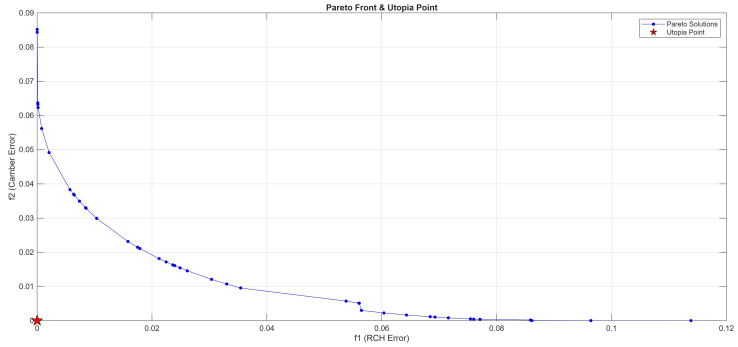


Figure 36

7 Optimization via Adams Insight

7.1 Balancer

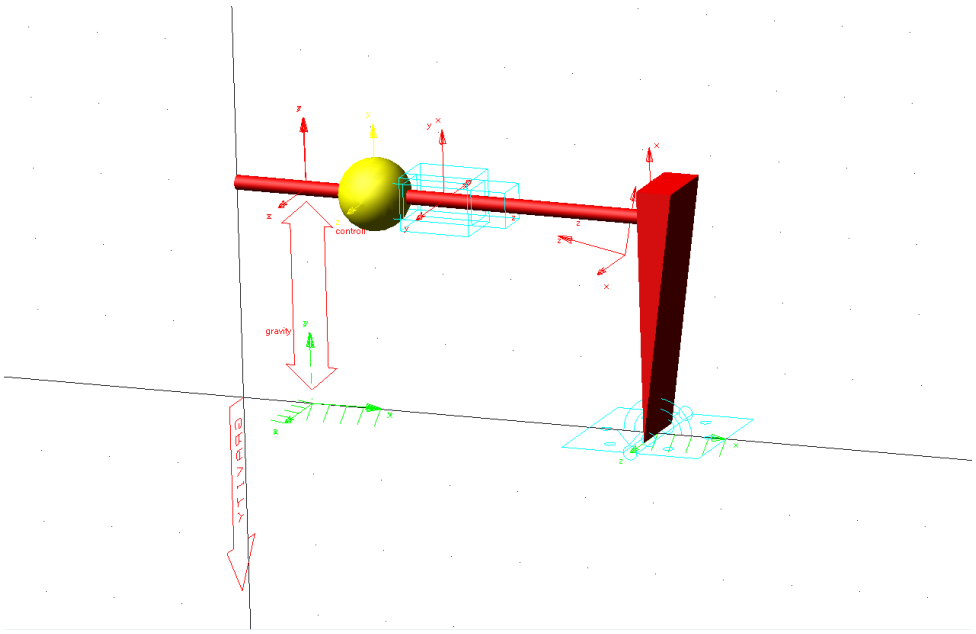


Figure 37: 3D mechanical model of the Balancer in Adams, illustrating the mass, hinge, and gravity/control force vectors.

The system under analysis is a mechanical **Balancer** shown in Figure 37. It consists of a rigid arm connected to a fixed base through a **revolute joint** (hinge). A spherical mass is attached to the arm, and the primary objective is to control its position over time. The control system must counteract the force of gravity using an opposing control force.

7.1.1 Design Variable

The control force is provided by a PID controller so the design variable are:

DV	Lower Bound	Upper Bound
$K_p x$	775	795
$K_d x$	416	424
$K_i x$	150.5	154.5

Table 9: Design Variables boundaries.

7.1.2 Objective Function

To evaluate the performance of the controller, a specific objective function was defined within Adams using a differential equation.The error function is defined as:

$$y' = \text{time}^3 \cdot \text{VARVAL}(\text{.Balancer.error})^2 \tag{24}$$

Where VARVAL is the distance measured directly in Adams between the center of the sphere and the target point on the arm. The use of the time^3 multiplier is a deliberate choice to **emphasize and penalize errors** that occur later in the simulation, ensuring that the system prioritizes long-term stability and steady-state accuracy. The initial condition for this objective is set to $y[t = 0] = 0.0$.

7.1.3 Optimization Strategy: DOE and Adams Insight

The optimization focused on tuning the PID controller gains (K_p, K_i, K_d) using a **Uniform Distribution**. The optimization was conducted in **Adams Insight** using a Design of Experiments (DOE) approach:

- **Polynomial Surface**: A 3rd-degree polynomial function was used to approximate the response.
- **Simulation Matrix**: To solve for a 3rd-degree polynomial with 3 design variables, the DOE required $4^3 = 64$ simulations.

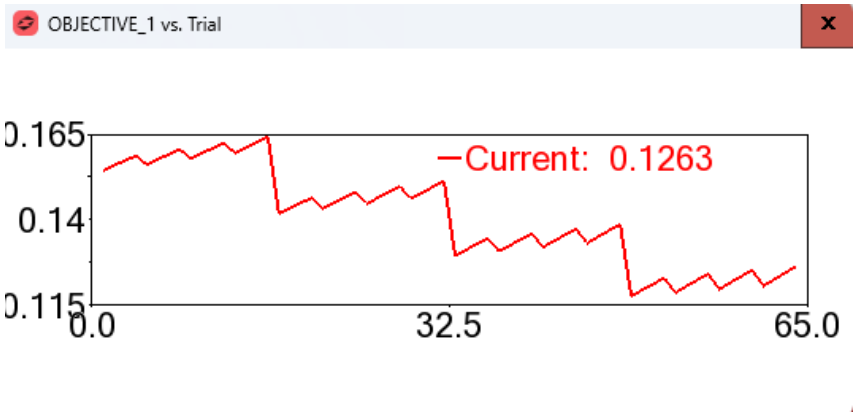


Figure 38Optimization progress: Objective_1 vs. Trial number

- **Convergence**: The optimization successfully reduced the objective value to a final "Current" value of **0.1263**.

The analysis of the trial history (Figure 40) provides significant insights into the sensitivity of the objective function regarding the PID parameters. The graph exhibits a characteristic "staircase" pattern, typical of a structured DOE where variables are adjusted across discrete levels.

From the observed trends, a clear correlation between the design variables and the system performance emerged:

- **Proportional Gain (K_p)**: To reach the minimum objective value of **0.1263**, the first variable (K_p) tends toward **maximization**. Physically, increasing the proportional gain enhances the "stiffness" of the controller, allowing it to counteract gravity more aggressively and reduce the steady-state error.
- **Integral and Derivative Gains (K_i, K_d)**: Conversely, the other two variables show a trend toward **minimization**. This suggests that while some integration is necessary to eliminate residual error, excessive K_i or K_d values might introduce unwanted oscillations or over-corrections. Given the cubic time-weighting (time^3)

in the objective function, such oscillations at the end of the simulation are heavily penalized.

This behavior confirms that the optimal configuration balances a high restorative force (high K_p) with minimal secondary corrections to ensure a smooth and stable convergence by the end of the 8-second window.

7.1.4 Results and Comparative Analysis

Adams Insight provides us with a tool that fitted the polynomial function so in this way tuning the design variable we have instantaneously the evaluation of the objective function, without simulating it. A final verification simulation was performed using the setup of the DVs shown in Figure 39.

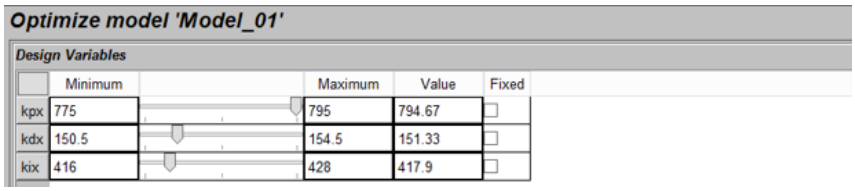


Figure 39: Adams Insight tool

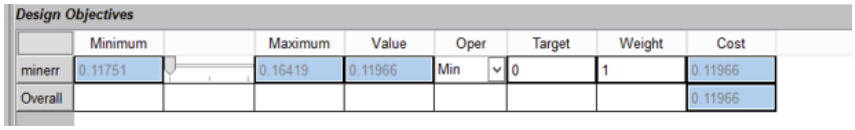


Figure 40: Objective polynomial function results

The final verification simulation confirms the success of the optimization. Figure 41 illustrates the system response over 8 seconds.The optimized "Last_Run" (dot blue line) demonstrates superior stabilization, effectively minimizing the error faster than the starting configuration.

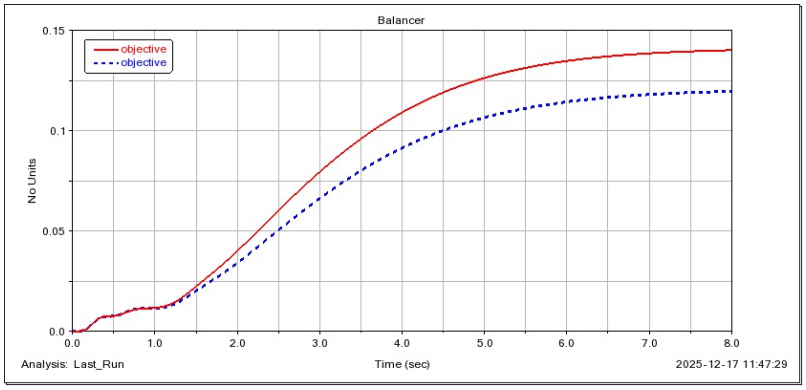


Figure 41

7.2 Case Study 2: Retractable Landing Gear Optimization

7.2.1 System Overview

The second analysis focuses on a retractable **Landing Gear** mechanism, as illustrated in Figure 42. The system is composed of a main strut, a wheel assembly, and an actuation linkage system consisting of upper and lower brackets. The primary goal is to optimize the kinematic geometry to ensure proper retraction without mechanical interference while minimizing the load on the actuator.

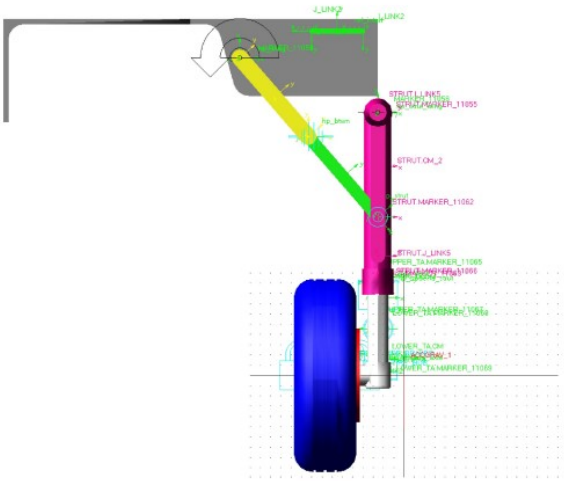


Figure 42Multibody model of the Landing Gear mechanism.

7.2.2 Design Variables

This study optimizes the **geometric properties** of the linkage. The selected Design Variables (DV) are:

- Length of the Upper Bracket
- Length of the Lower Bracket

DVs	Lower Bound	Upper Bound
Lower Bound	39	57
Upper Bound	40	58

Table 10: Design Variables boundaries.

7.2.3 Optimization Objectives

The optimization problem in Adams Insight is set up as a multi-objective study:

1. **Interference Avoidance (Clearance):** The primary constraint is to prevent any compenetration between the landing gear components and the aircraft frame. The objective function monitoring the clearance distance was set to a specific **target value of -10 cm**.

2. **Torque Minimization:** The second objective is to minimize the **Maximum Torque** required by the actuator during the retraction cycle, thereby optimizing the energy efficiency of the system.

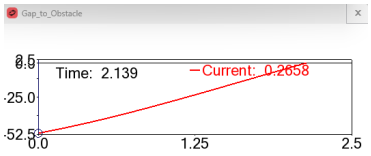
7.2.4 Preliminary simulations

Before applying the DOE strategy, three simulations are set, maintaining the *Lower Bracket Length* fixed and varying the other Design Variable.

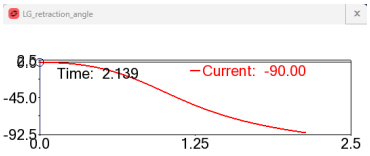
simulation	Lower Bracket	Upper Bracket
1	51	51
2	51	40
3	51	58

Table 11: Design Variables boundaries.

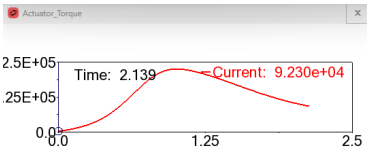
The first simulation provides us with the following results:



(a) gap

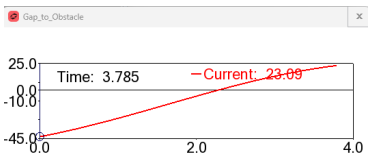


(b) retraction angle

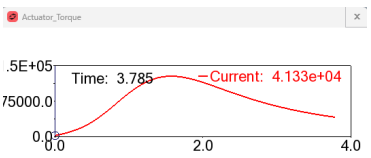


(c) Torque

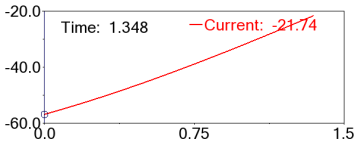
For the second and the third ones if we focus on the maximum torque value and the gap:



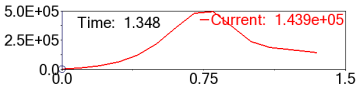
(a) gap



(b) Torque



(a) gap

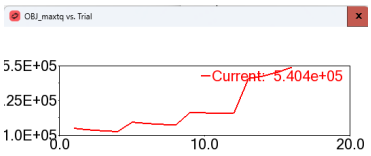


(b) Torque

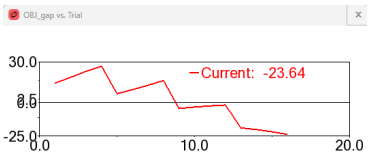
As we can see in the second simulation the gap is positive, so we have compenetration while in the third the Maximum of the Torque is very high $1,439e+05$

7.2.5 Adams Insight

Following a methodology similar to the previous case, an initial baseline simulation was performed, followed by a DOE analysis within Adams Insight. The solver sought to balance the two competing objectives (Torque vs. Clearance) by adjusting the bracket lengths within the bell curve of the **Gaussian distribution**. Also for this DOE analysis the polynomial function was a $3^r d$ order, so $4^3 = 64$ simulations are required.



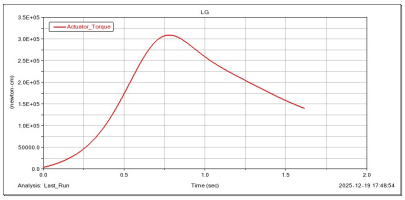
(a) Max torque value vs trials



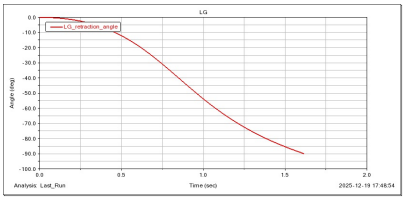
(b) Gap vs trials

After evaluating some configurations through the polynomial function a last simulation with the following DV value:

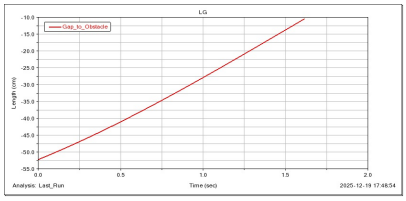
- Length of the Upper Bracket: $54.7mm$
- Length of the Lower Bracket $42,5mm$



(a) Torque.



(b) Retraction angle.



(c) Gap.

As we can see Max Torque is maintained sufficiently low and the gap is in the safety region so the DOE analysis and the entire optimization process can be considered successful It is important to note that the **Retraction Angle** was monitored during the standard simulations to ensure the gear reaches the stowed position and to evaluate also the retraction time:we also obtain an improvement under this aspect compared with the first simulation $1,64s$ vs $2,1s$. However, this parameter was treated as a check value and was **not included** as an active variable in the optimization loop.

7.3 Ferrari Testarossa Suspension Analysis (Adams Car)

7.3.1 Parallel Wheel Travel Simulation

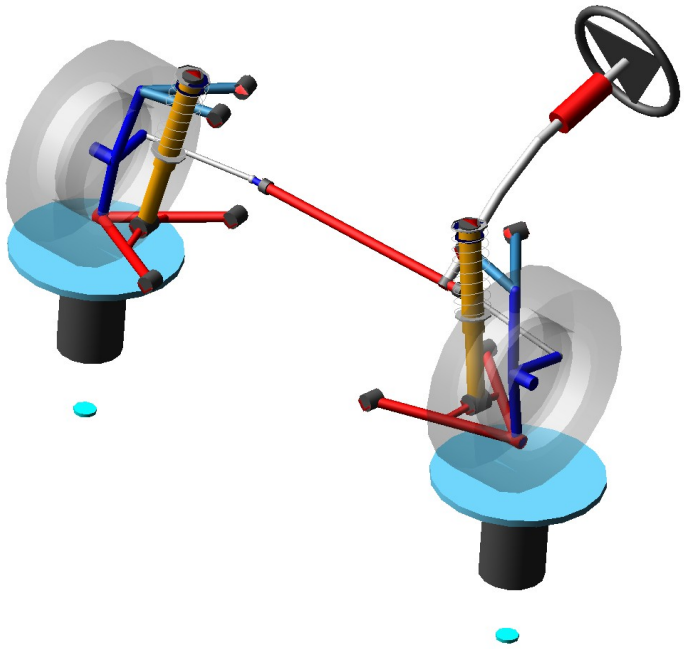


Figure 48

The kinematics of the front suspension were analyzed using a *Parallel Wheel Travel* maneuver in Adams Car. The simulation parameters were set to evaluate the suspension behavior under vertical excitation:

- **Bump Travel:** +90 mm
- **Rebound Travel:** −90 mm
- **Steps:** 90

The initial objective was to reduce the distance between the maximum and the minimum value of*Toe Angle* without varying too much the *Camber Angle* throughout the wheel travel.

7.3.2 DOE Setup in Adams Insight

To find an optimal compromise between Toe and Camber characteristics, a Design of Experiments (DOE) was set. Four geometric coordinates were promoted as Design Variables (candidates) for the optimization. A *Uniform Distribution* was assigned to all variables with a variation range of ±5 mm around the nominal value:

Variable Name	Hardpoint Component	Nominal [mm]	Range
Lca_x	hpl_lca_outer (x)	270	±5
Lca_y	hpl_lca_outer (y)	-760	±5
Lca_z	hpl_lca_outer (z)	130	±5
$Tierod_z$	hpl_tierod_outer (z)	330	±5

Table 12: Design Variables for Suspension Optimization.

The strategy employed was a *Full Factorial* design with a quadratic model:

- **Design Type:** Screening (2 level) / Full Factorial
- **Number of Runs:** $3^4 = 81$ runs (3 levels for 4 variables).
- **Objectives:** Minimize the maximum absolute value of both Toe Angle and Camber Angle during the simulation.

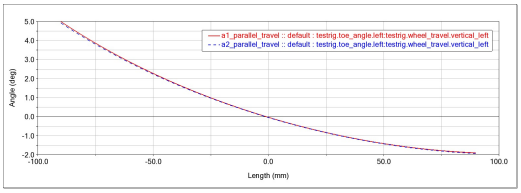
7.3.3 Results

The optimized hardpoint coordinates extracted from Adams Insight are:

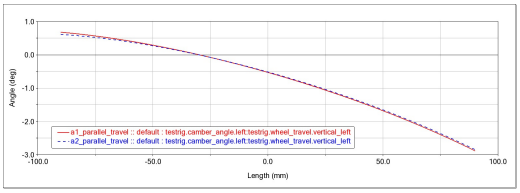
- $Lca_x = 266.8$ mm
- $Lca_y = -765$ mm
- $Lca_z = 134.4$ mm
- $Tierod_z = 325.17$ mm

And a full simulation in Adams Car was run The optimization resulted in the following performance metrics:

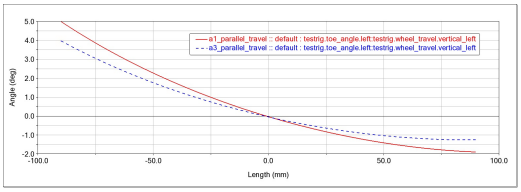
- **Optimized Max Toe:** 3.97 deg
- **Optimized Max Camber:** 0.42 deg



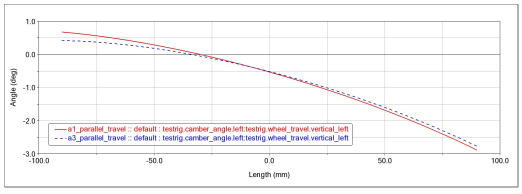
(a) Toe Angle vs Wheel Travel comparison.



(b) Camber Angle vs Wheel Travel comparison.



(c) Toe Angle vs Wheel Travel comparison.



(d) Camber Angle vs Wheel Travel comparison.

7.4 Full Vehicle Analysis: Step Steer Optimization

Following the suspension refinement, the analysis was extended to the **Full Vehicle Model** of the Ferrari Testarossa (Figure 50). The objective was to evaluate the vehicle's dynamic response during an open-loop steering event.

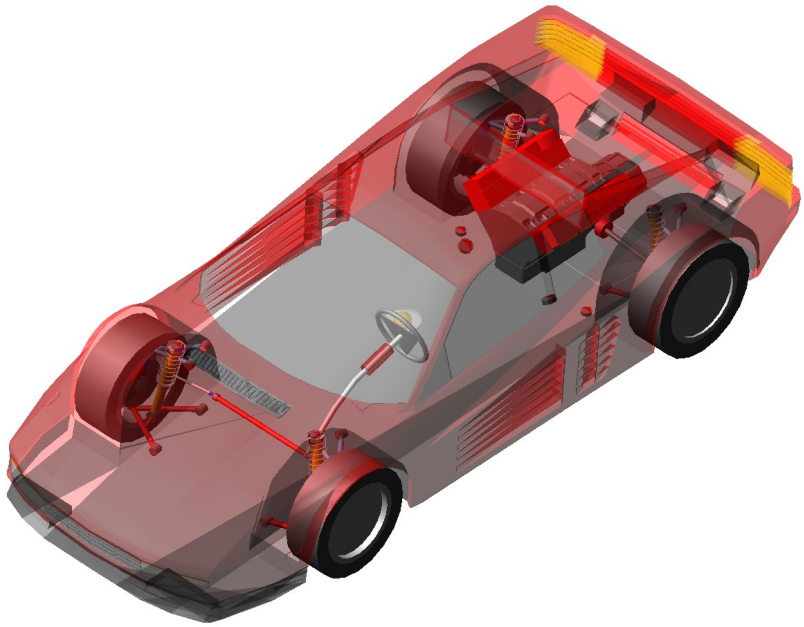


Figure 50: Full vehicle model of the Ferrari Testarossa in Adams Car.

The selected maneuver was a **Step Steer** with the following parameters:

- **Velocity:** 100 km/h
- **Steering Input:** Final value of 45 deg
- **Timing:** Step start at 1.0 s, duration 0.5 s
- **Simulation Length:** 6.0 s (600 steps)

7.4.1 Optimization Setup

The optimization utilized the same four hardpoint coordinates defined in the suspension analysis (LCA x,y,z and Tierod z). However, a crucial methodological change was applied to the design variables:

- **Distribution:** Unlike the uniform distribution used previously, a **Normal Distribution** was assigned to the variables to better represent realistic manufacturing statistical variations.

7.4.2 Objectives

The goal was to minimize three key dynamic indicators:

1. **End Steering Angle:** To improve steering response efficiency.
2. **Max Roll Angle:** To increase stability and passenger comfort.
3. **Max Yaw Rate:** To ensure predictable handling.

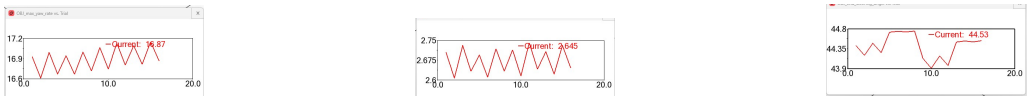
7.4.3 DOE Methodology and Results

A DOE based on an interaction model (Linear) was executed. With 4 variables and a 2-level design, the matrix consisted of $2^4 = 16$ runs.

The results, displayed in Figure 51, highlight a trade-off inherent in vehicle dynamics. It was observed that the configuration minimizing the steering angle does not correspond to the optimal values for roll and yaw.

The theoretical "Utopia Point" (the ideal but unreachable combination of best individual results) was identified as:

- Max Yaw Rate: 16.61 deg/s
- Max Roll Angle: 2.60 deg
- End Steering Angle: 43.90 deg



(a) Max Yaw Rate (b) Max Roll Angle (c) End Steering Angle

Figure 51: DOE Trial results for the three optimization objectives.

8 Lab 8: Neural Network Application on a 2-Variables Function

8.1 Problem definition:

The aim of this laboratory is to evaluate the capability of a Feedforward Neural Network to learn and generalize a non-linear two-variable function.
The target function to be approximated is defined in the domain $x, y \in [-2, 2]$ as:

$$Z = X \cdot \exp(-X^2 - Y^2)$$

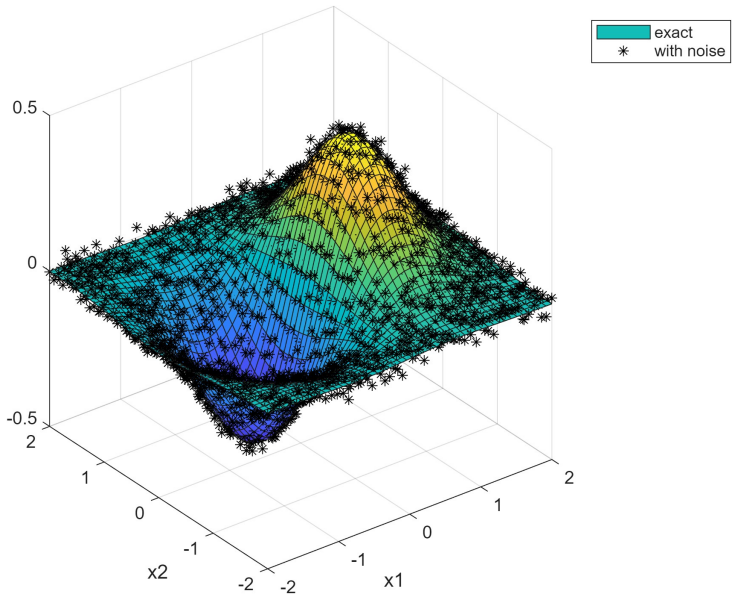


Figure 52

To simulate a real-world scenario, the training data was generated by adding a Gaussian noise component to the exact values of the function:

$$F_{noise} = F_{exact} + \text{noise}$$

To assess the network's quality, two Mean Squared Error (MSE) values were compared:

- Noise MSE (MSE_{noise}): The error between the "noisy" training data and the exact function. The Noise MSE was evaluated considering the following data set:

```
x1 = linspace(-2,2,50);
x2 = linspace(-2,2,50);
dv = combvec(x1,x2)';

fexact = dv(:,1).*exp(-dv(:,1).^2 - dv(:,2).^2);
fnoise = fexact + 0.02*randn(size(dv,1),1);

MSE_noise = mean((fexact - fnoise).^2);
```

$$MSE_{noise} = 4.0439e-04$$

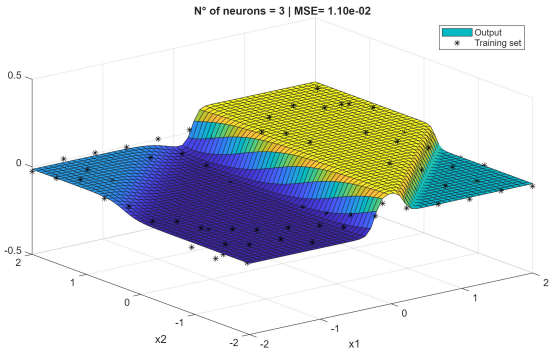
- Network MSE ($MSE_{network}$): The error between the neural network's output and the exact function.

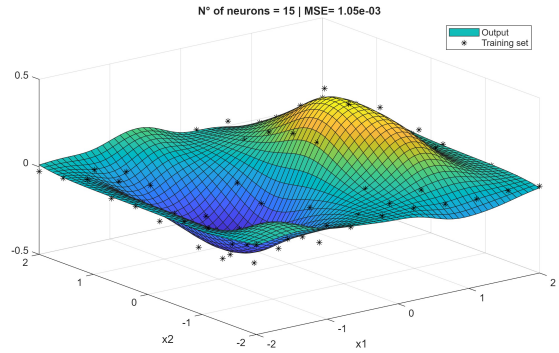
8.2 Research on Network Parameters influence:

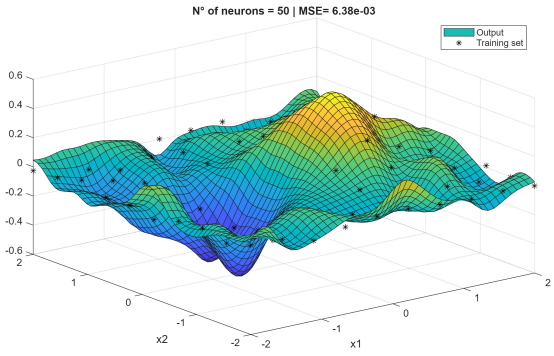
8.2.1 Network Capacity (Number of Neurons):

Evaluate how the number of neurons in the hidden layer affects the network's ability to learn the function and the risk of overfitting.

Parameter	Value
Training points	100
n° neurons	[3 15 50]
n° hidden layers	1
Training algorithm	'trainlm'
Training ratio	0.8
Validation ratio	0.2







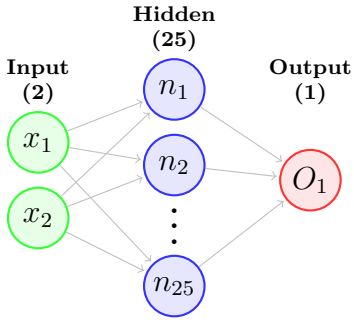
- **Comments on results:**
  - With **3 neurons**, the network suffers from Underfitting: the surface is too rigid and fails to capture the curvature of the central bell, resulting in a high mean square error
  - With **15 neurons**, the best compromise is achieved: the reconstructed surface is faithful to the original function and smooth
  - With **50 neurons**, the phenomenon of Overfitting (or Overtraining) occurs: the network learns exactly the noisy input data instead of modeling the shape of the exact function.

8.2.2 Architecture Efficiency (Number of total Weights):

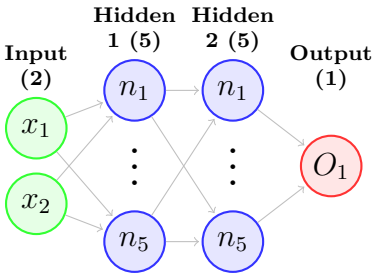
Verify whether a deep architecture (multi-layer) is more efficient than a wide one (Single-layer) in approximating the function.

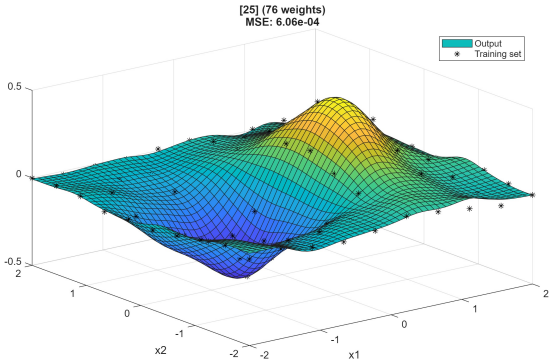
Parameter	Value
Training points	100
Training algorithm	'trainlm'
Training ratio	0.8
Validation ratio	0.2

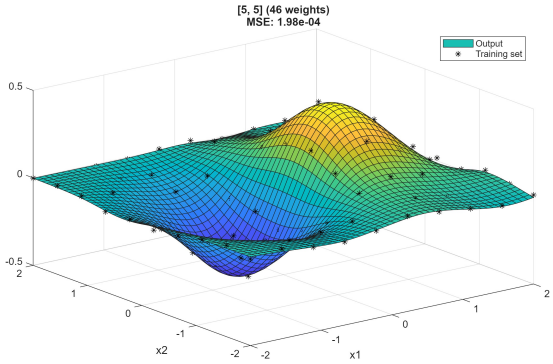
One Hidden Layer (25 Neurons)



Two Hidden Layers (5+5 Neurons)





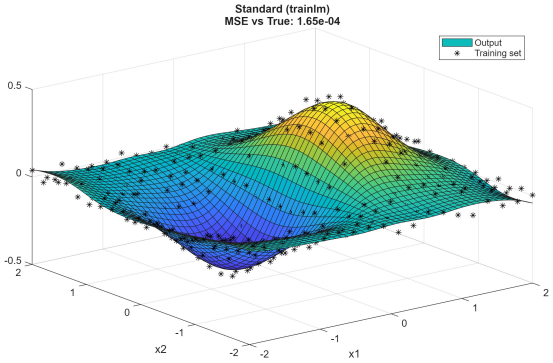


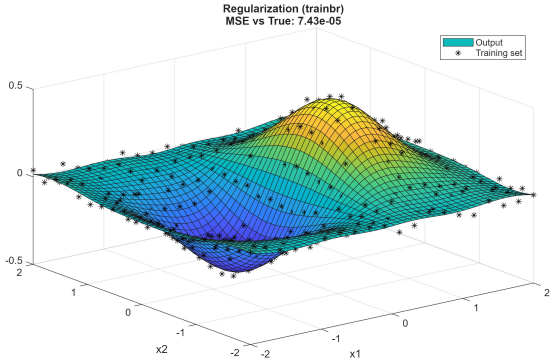
- **Comments on results:** The experiment confirms that the multi-layer architecture (46 weigths) is capable of achieving a lower MSE and a better approximation than the one-single layer architecture (76 weigths) even if the total number of neurons is lower.

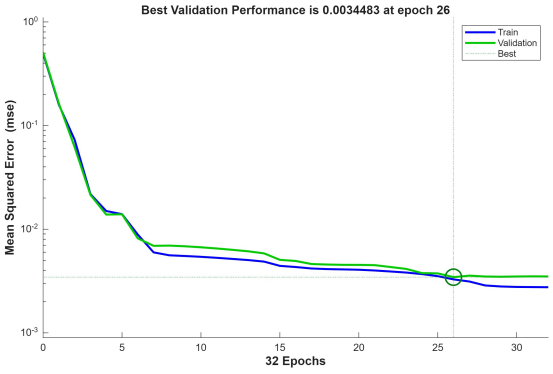
8.2.3 Training algorithms (Standard vs Regularization):

Compare the robustness of different training algorithms in the presence of highly noisy data and scarcity of validation data.

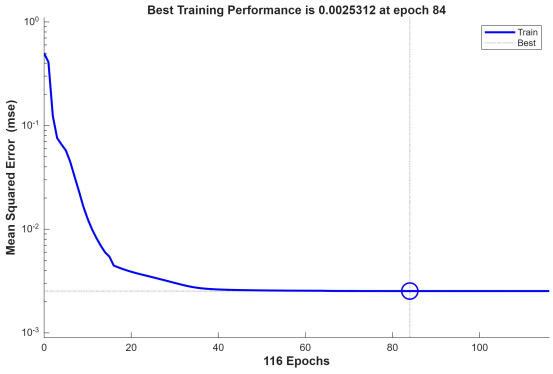
Parameter	Value
Training points	400
Training algorithm	['trainlm' 'trainbr']
Training ratio	0.8
Validation ratio	0.2







Levenberg-Marquardt ('trainlm')



Bayesian Regularization ('trainbr')

- **Comments on results:** The Bayesian Regularization Training algorithm doesn't need to store some of the training points for computing the Validation error; instead, it applies a regularization by modifying the mean square error function ($MSEreg = \gamma mse + (1-\gamma)msw$), penalizing high-value weights. This forces the network to produce a "smoother response", effectively filtering the noise and recovering the underlying shape of the function even in critical conditions.

8.3 Final Network Design:

All the knowledge acquired in the previous study cases were used to build a more efficient Neural Network able to perfectly reproduce the function and filter the noise component: the Network works with an optimized number of neurons, distributed on multiple layers to reduce the total number of weights. A wider set of training data was provided and the Bayesian Regularization Training Algorithm was employed to avoid overfitting:

Parameter	Value
Training points	2500
Training algorithm	'trainbr'

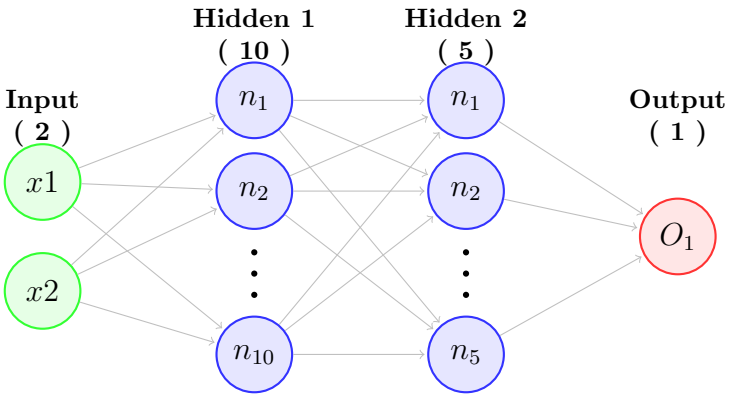


Figure 57

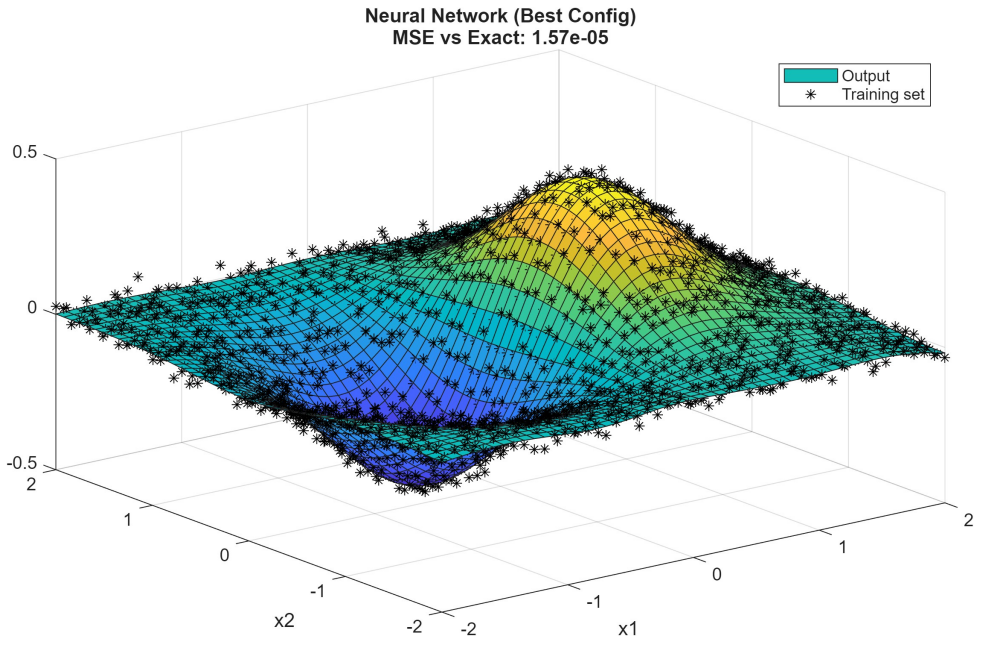


Figure 58

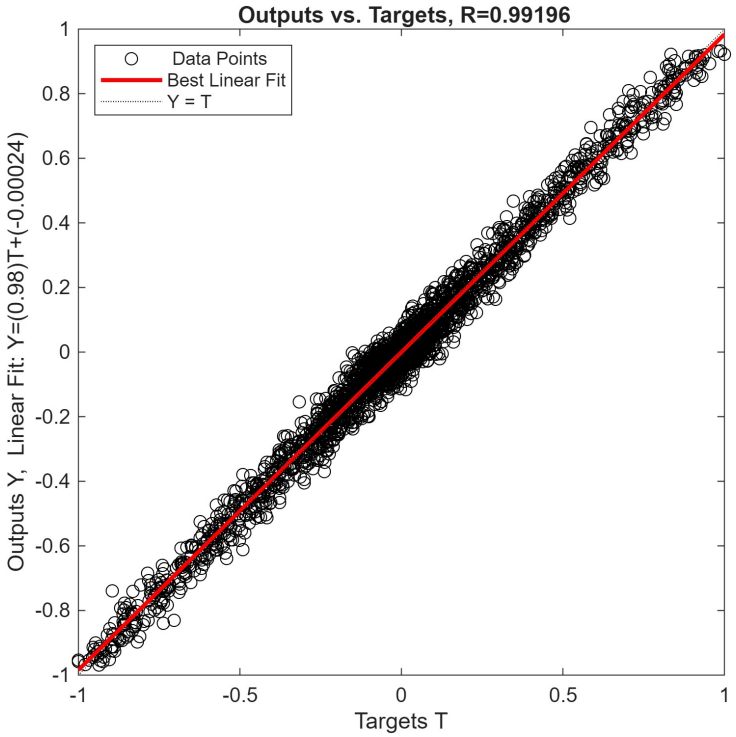


Figure 59

- **Comments on results:** The Mean Square Error evaluated between the Output of the Network and the exact function shows a quasi-perfect alignment of the two curves.

Parameter	**Value**
$MSE_{network}$	1.57e-05
MSE_{noise}	4.0439e-04

9 Lab 9: Vehicle Handling Optimization

9.1 Problem definition:

The primary goal of this laboratory is to optimize the handling performance of a passenger car by tuning specific parameters related to the vehicle layout and suspension system. The optimization process relies on three performance indices extracted from two specific open-loop maneuvers:

- **Steering Pad constant speed maneuver:** performed under quasi-stationary conditions to characterize the handling capability and the linearity of the vehicle's response.
- **Throttle Release during cornering:** it simulates a transient condition starting from a steady state cornering. It evaluates the vehicle's stability following a sudden load transfer and grip variation.

The optimization procedure was carried out by training a Neural Network able to approximate the relationship between the design variables and the objective functions.

9.2 Design Variables and Objective Functions:

- **Design Variables:**
  - Center of Gravity position (a):
    Defines the longitudinal position of the center of gravity measured from the front axle. It determines the static load distribution between the front and rear axles.
  - Front Anti-Roll Bar Stiffness (H):
    Represents the equivalent stiffness of the front anti-roll bar. Varying this parameter alters the lateral load transfer of the front axle and can affect the understeer/oversteer tendency of the vehicle.

Parameter	**Range value**
a [mm]	[700 850]
H [-]	[0.6 1.2]

- **Objective Functions:** Three objective functions are defined to quantify the handling performance, derived from the maneuvers described above.
  - Maximum Lateral Acceleration:
    Derived from the Steering Pad maneuver, this index corresponds to the peak value of the handling diagram. The goal is to maximize it in order to ensure high cornering performance.
    Since the optimization aims to minimize all the objectives, the negative value of the lateral acceleration is considered.

– Handling Diagram Curvature:
Also derived from the Steering Pad, this index is defined as the absolute value of the peak of the second derivative of the handling curve. The goal is to minimize this value to obtain a more progressive and smooth vehicle response.

– Yaw Rate Variation:
Derived from the Throttle Release maneuver, this index measures the deviation of the yaw velocity ($\dot{\psi}$) 1 second after the throttle is released ($t_n = t_0 + 1s$) compared to the stationary condition value ($\dot{\psi}_0$). The goal is to minimize this variation to ensure vehicle stability and prevent excessive oversteer reactions during load transfer.

9.3 Correlation Matrix:

Before training the Neural Network, a correlation search was conducted: the purpose of this analysis is to evaluate potential redundancies among the three objective functions defined for the vehicle optimization problem

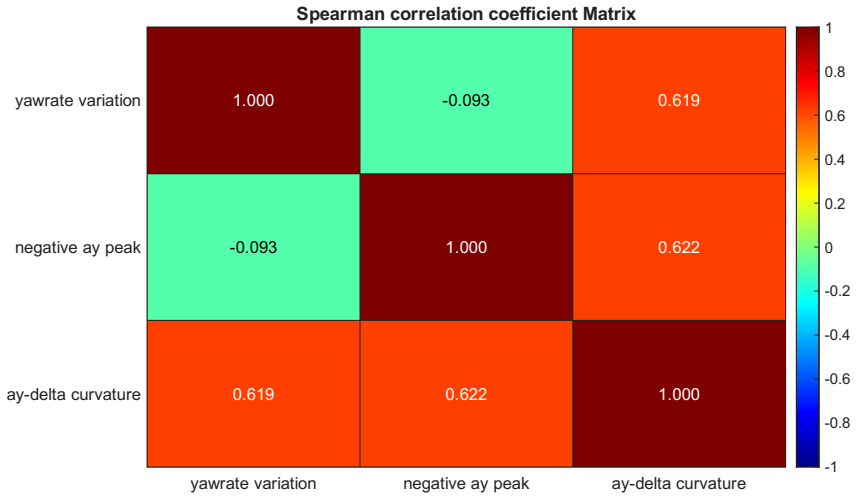


Figure 60

The correlation matrix shows complete uncorrelation between the Yawrate Variation and the Peak of Negative Acceleration objective functions, meaning that the two will surely conflict each other during the optimization process. Regarding the other two combinations of objectives, the matrix shows a Spearman correlation coefficient of approximately 0.6 which suggests a possible correlation, but does not fully guarantee a strong dependency. For this reason, to rigorously quantify how one objective predicts another, the Coefficient of Determination (R^2) was computed via linear regression:

Objectives	R^2
Yaw Rate vs Ay Peak	0.0035
Ay Peak vs Curvature	0.4328
Yaw Rate vs Curvature	0.3695

The analysis confirms that even if the Spearman correlation coefficient may lead to a 2-objective optimization problem, the Coefficient of Determinations confirms that the dependency is too weak and so the final choice is a 3-objective optimization problem.

9.4 Neural Network Training:

The Neural network was trained on a set of input data which are simulations outputs carried on different a and H combinations. The following architecture was implemented:

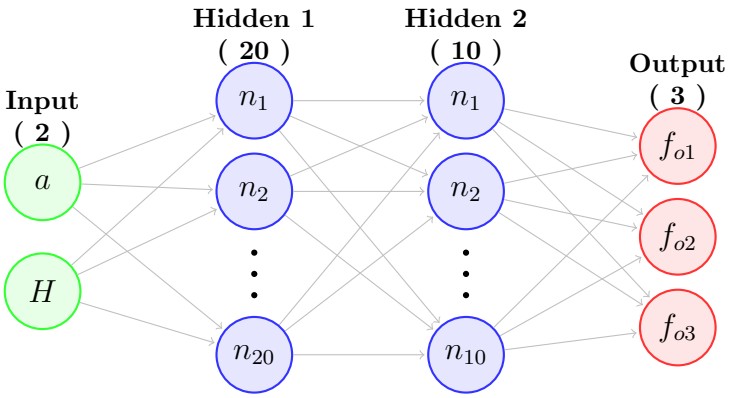
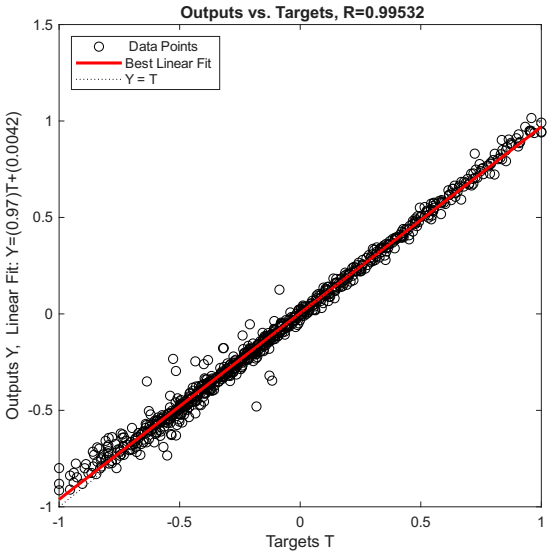
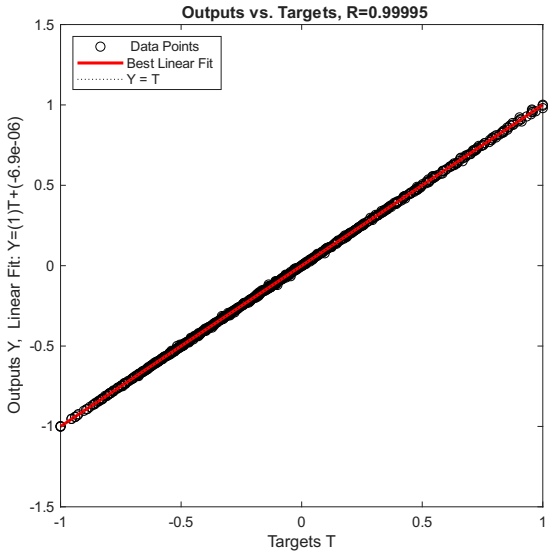


Figure 61: Neural Network Diagram

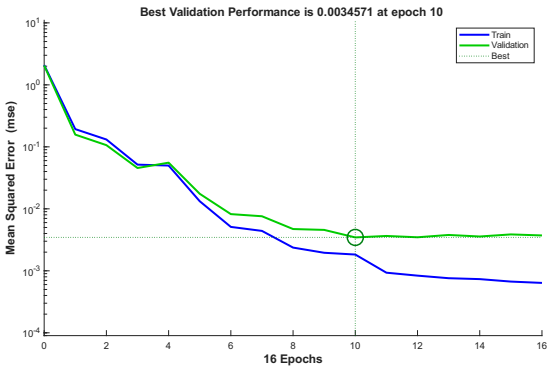
The following plots represent a comparison between the performance of the Neural Network with a Levenberg-Marquardt ('trainlm') training algorithm implemented and the one of the same Neural Network but with the Bayesian regularization ('trainbr') training algorithm:



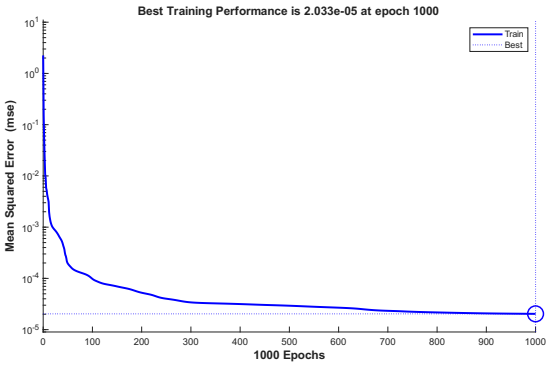
Levenberg-Marquardt ('trainlm')



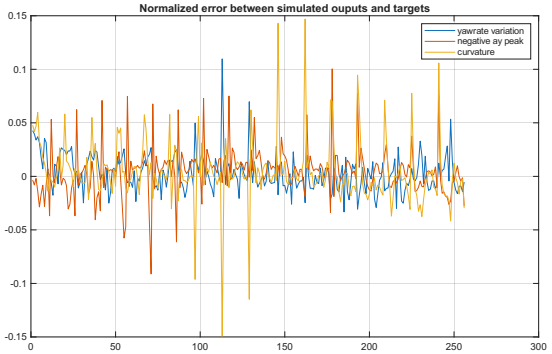
Bayesian Regularization ('trainbr')



Levenberg-Marquardt ('trainlm')

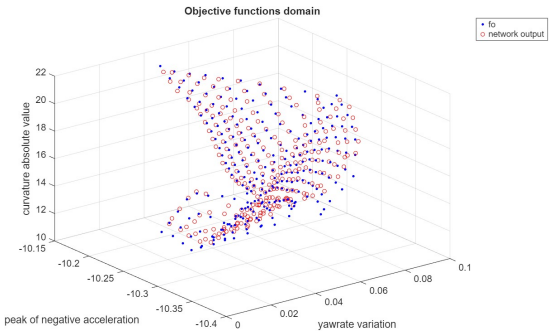


Bayesian Regularization ('trainbr')

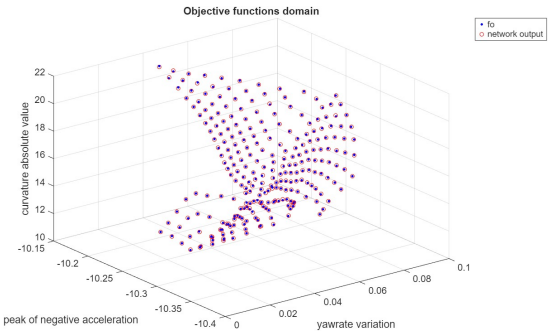


Levenberg-Marquardt ('trainlm')

Bayesian Regularization ('trainbr')



Levenberg-Marquardt ('trainlm')



Bayesian Regularization ('trainbr')

Comparison of the results shows that 'trainbr' outperforms 'trainlm' by providing a more robust approximation of the training points. Its built-in regularization mechanism automatically limits model complexity, allowing for a tight fit without the risk of overfitting, effectively filtering out simulation noise.

9.5 Optimization Procedure:

9.5.1 Pareto Optimal Set:

The 3-Objective optimization problem was performed considering the Neural Network trained with the 'trainbr' algorithm. The Pareto Optimal set was computed by considering a simulation of the network on 10000 samples extracted from a uniform grid in the design variables domain.

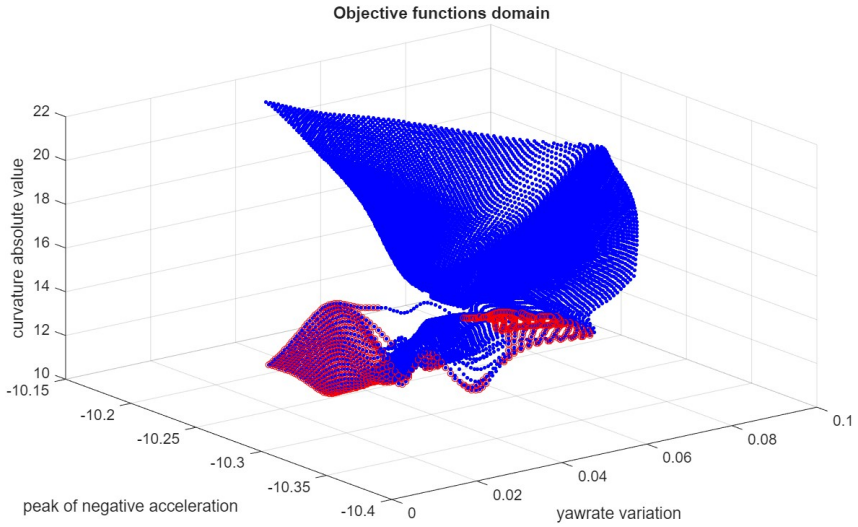
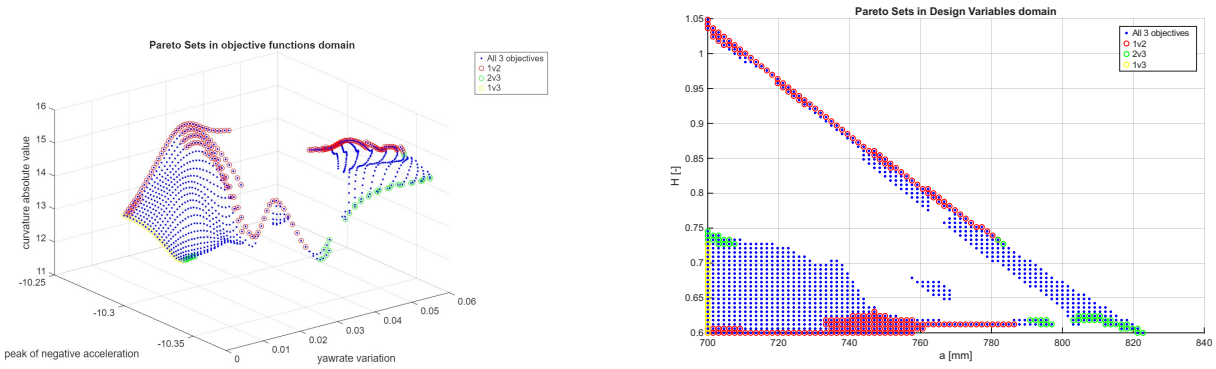


Figure 66

Figure 67

9.5.2 Choice of the Best Solution:



To select a final optimal solution among the ones of the 3D Pareto front, the **k-optimality criterion** was first evaluated. The analysis of the dataset confirmed that no point within the discrete frontier meets the strict requirements for k-optimality (for k >= 1), as no single solution dominates others across all objective pairs.

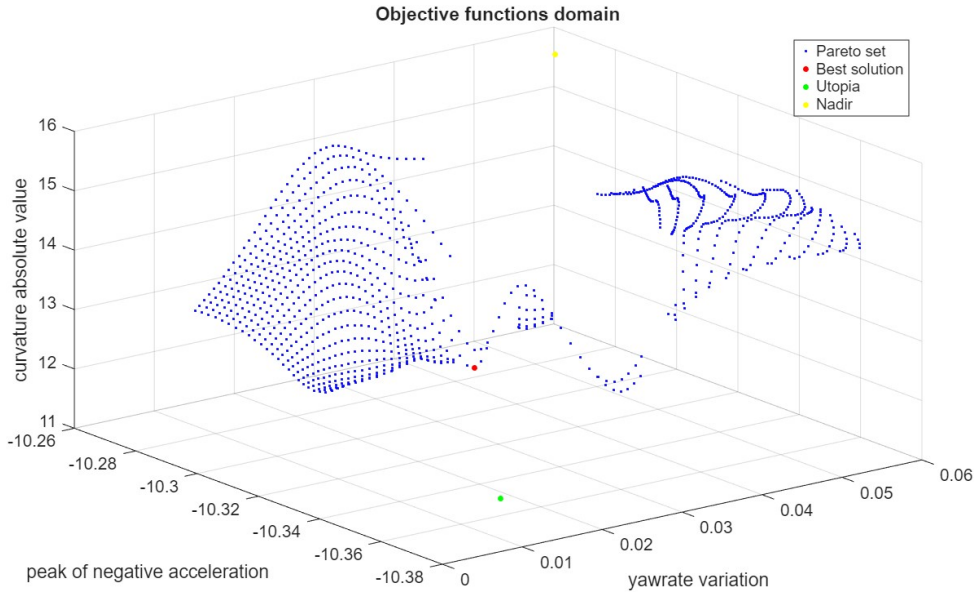


Figure 69

Therefore, the **"Closest to Utopia"** method was adopted as the final criterion. The three objective functions (Yaw Rate, Acceleration, and Curvature) were normalized to a [0, 1] range to handle their different physical scales. The design point with the minimum Euclidean distance to the ideal Utopia point was identified as the best compromise solution, ensuring a balanced minimization of all performance metrics.

	a [mm]	**H [-]**
Best solution	735	0.7

9.5.3 Simulation:

The Model was then updated with the new optimal design variables configuration and analyzed on VI-car with respect to the standard configuration ($a = 1200$ mm, $H = 1$) obtaining significant improvements for what concerns the vehicle behavior for both the Steady state steering Pad and the Power off maneuvers:

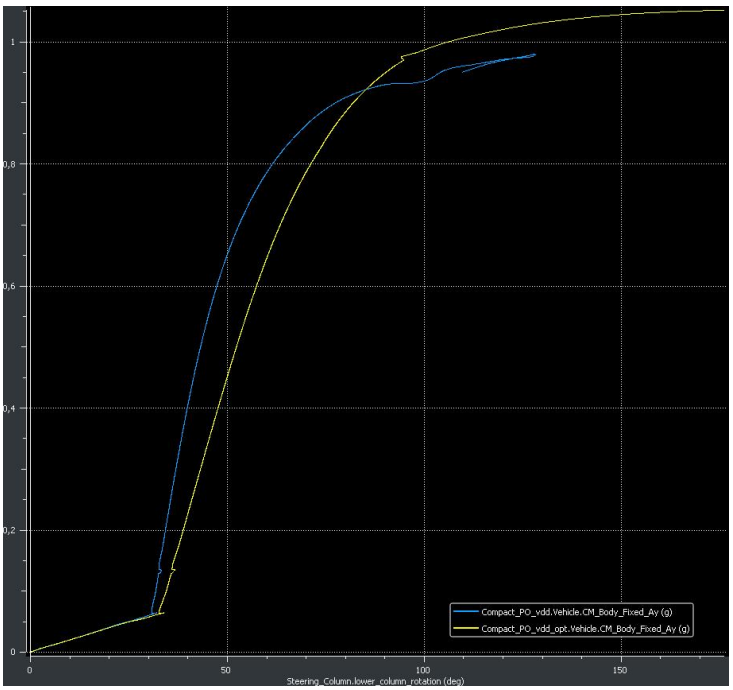


Figure 70: Ay vs δ diagram

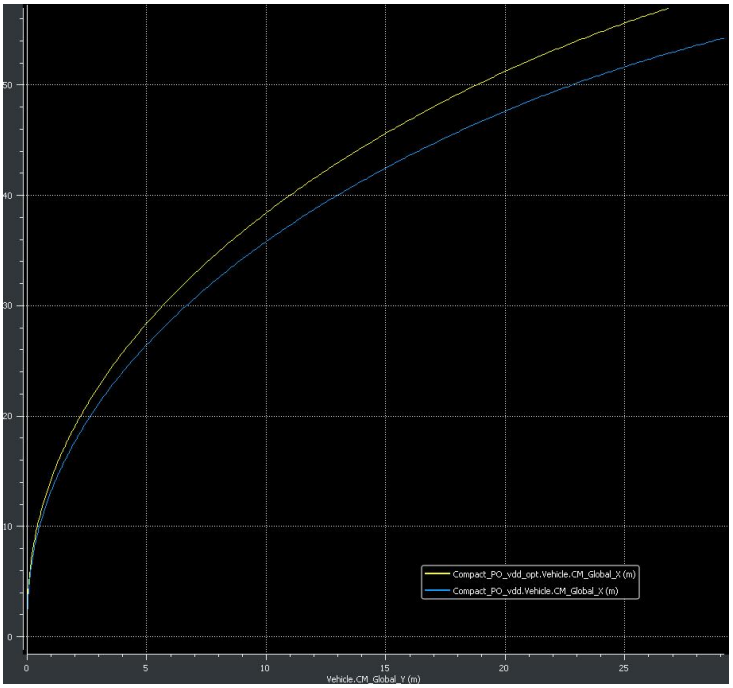


Figure 71: Power off maneuver trajectory

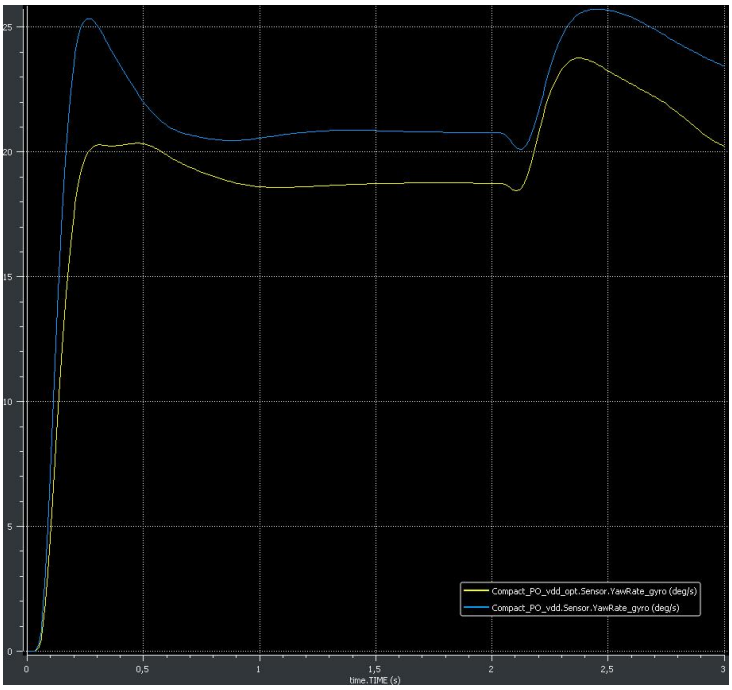


Figure 72: $\dot{\psi}$ vs time diagram

The yellow line represents the simulations results of the vehicle adjusted with the Best optimal solution parameters: it is possible to notice the improvements for the maximum handling capability and the transition from the linear state to the unstable one, but also the ability of the vehicle to better react when the power is switched off.

10 Lab 11: Structural Optimization Laboratory

10.1 Matlab 99 line optimization code

10.1.1 Problem definition:

The aim of this section is to understand better how the Topology optimization works by looking at the Ole Sigmund "99 Line Code". This compact MATLAB implementation solves the minimum compliance problem (maximizing stiffness) for a simplified 2D continuum structure using the SIMP (Solid Isotropic Material with Penalization). The method relies on a power-law penalization to bring the solution towards a more "binary" distribution (solid/void), making it physically interpretable and manufacturable.

10.1.2 Experimental Procedure:

To fully understand the mechanism of the algorithm, a sensitivity analysis was conducted. The investigation focused on the impact of four main governing parameters:

- **Volume Fraction (volfrac):** to quantify the trade-off between mass reduction and structural stiffness
- **Penalization Factor (penal):** to analyze the convergence from "grey" intermediate solutions to a more "black and white" one.
- **Filter Radius (rmin):** to understand its role in imposing a minimum length scale constraints and preventing numerical instabilities like checkerboarding
- **Mesh Refinement (nelx, nely):** A specific study was performed to verify mesh independence; to guarantee this phenomenon, a proper scaling od the filter radius proportionally to the mesh density was needed.

A One-Factor-at-a-Time (OAT) approach was used to isolate the influence of each single design variable. The procedure started by first considering a baseline configuration with standard parameter values:

Parameter	Value
volfrac	0.5
penal	3
rmin	1.5
nelx	100
nely	20

Each single parameter was varied across a defined range while keeping the others fixed at their baseline values.

10.1.3 Results and comments:

- **Volume fraction effect:**
  - Effect on Topology:

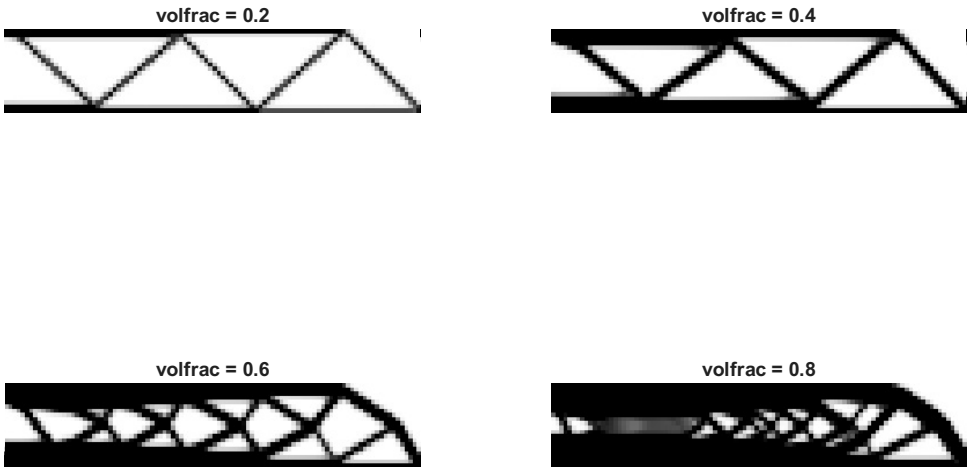


Figure 73

For a low $volfrac$ value, the algorithm produces a truss-like structures characterized by thin members optimized to transfer loads directly to the supports. For a high $volfrac$ value, the algorithm procudes a more massive and compact structure

  - Effect on Compliance:

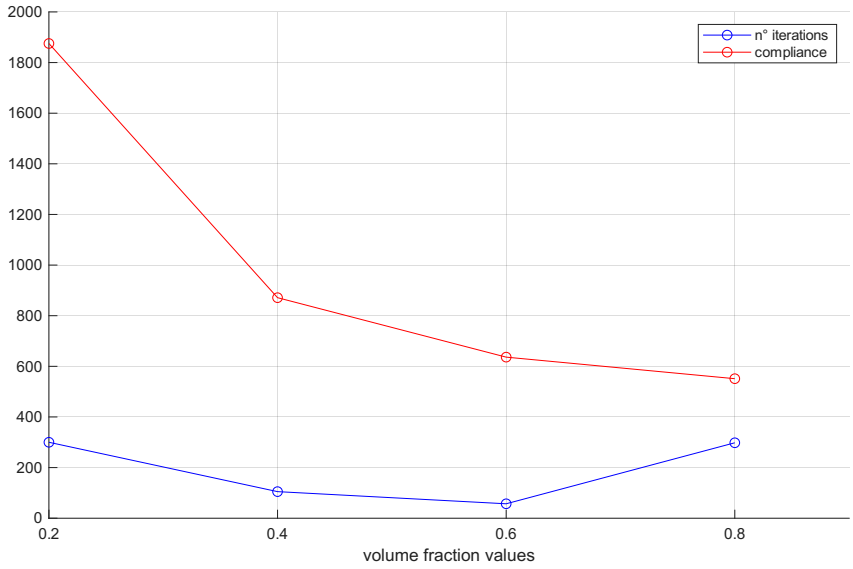


Figure 74

There is an inverse relationship: increasing the available material significantly reduces compliance (increase stiffness), as there is more material to distribute the stress. Consequently, low volume fractions results in highly flexible structures.

- **Penalization effect:**
  - Effect on Topology:

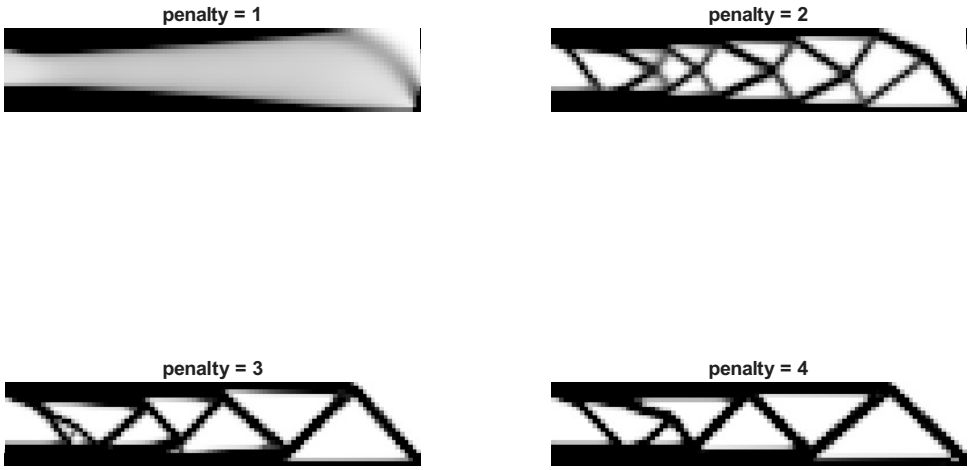


Figure 75

For low *penal* value, the structure resembles a cloudy density map rather than a distinct physical shape. For a high *penal* value, intermediate densities become structural inefficient. The algorithm is forced to choose between 0 (void) or 1 (solid), resulting in a clear "black and white" design suitable for manufacturing

  - Effect on Compliance:

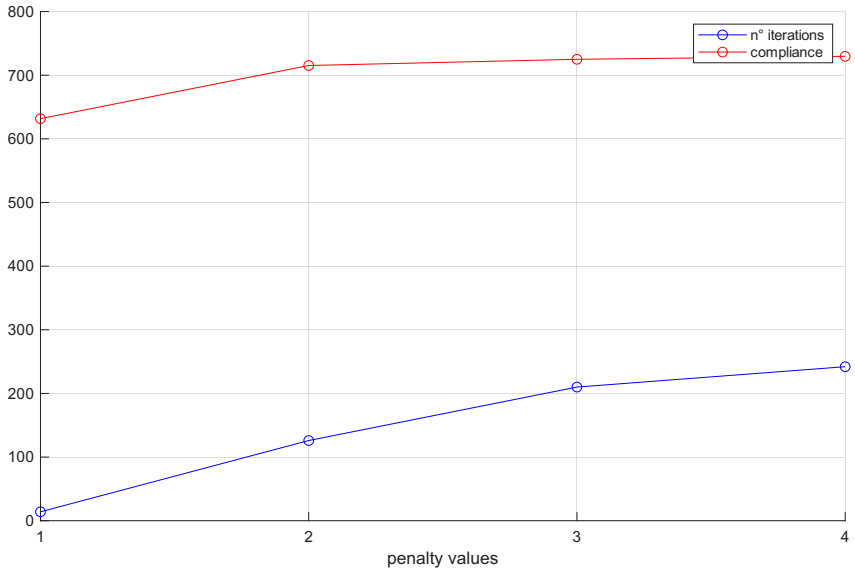


Figure 76

Compliance increases as penal increases: this is because the penalization mathematically "devalues" the stiffness of intermediate densities elements. While a design with penal = 3 is physically more realistic, it is numerically "less stiff" than the theoretical (but impossible) grey solution allowed by *penal* = 1

- **Filter Radius effect:**
  - Effect on Topology:

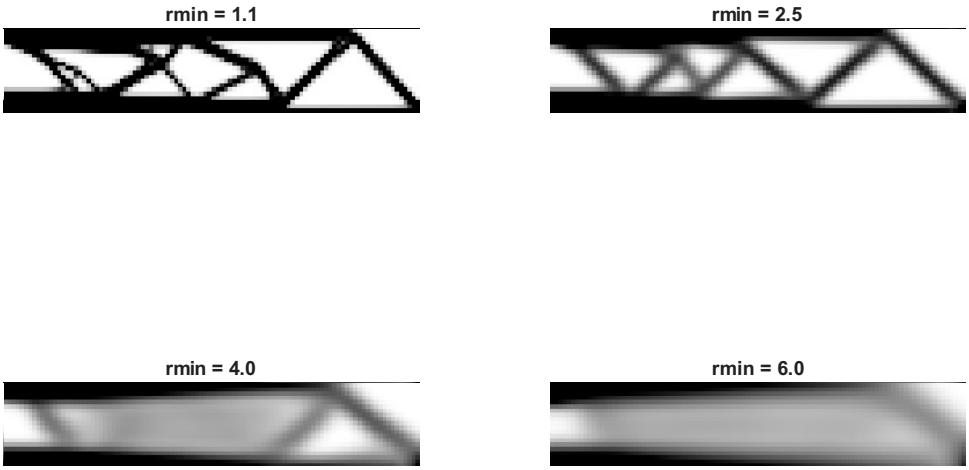


Figure 77

For a low *rmin* value, the constraint on member size is weak. This leads to very fine, detailed micro-structures but often results in numerical instabilities such as checkerboarding. For a high *rmin* value, the filter imposes a larger minimum length scale. Fine details vanish, and the solution becomes thicker and simpler. However, if *rmin* is too high we could also have situation where the transition from solid to void is too large and so grey areas may appear

  - Effect on Compliance:

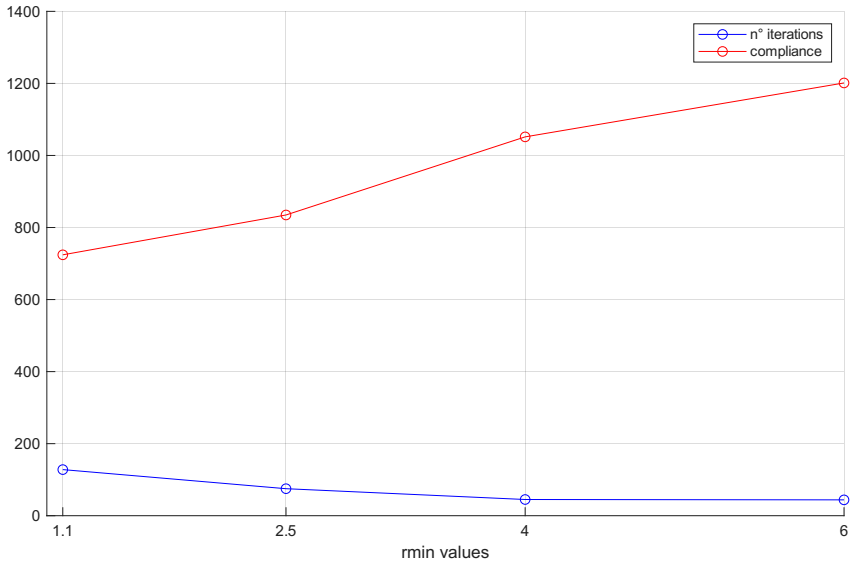


Figure 78

Compliance increases as *rmin* increases: by enforcing a larger minimum feature size, the filter creates wider "grey" transition boundaries which, under high penalization (like penal = 3) are structurally inefficient, thus reducing the overall stiffness of the final design

- **Mesh Size effect:**
  The discretization of the design domain was analyzed by varying the number of finite elements (*nelx* and *nely*) while maintaining a constant aspect ratio (5:1) and keeping the physical size of the filter constant (by scaling *rmin* proportionally)

  – Effect on Topology:

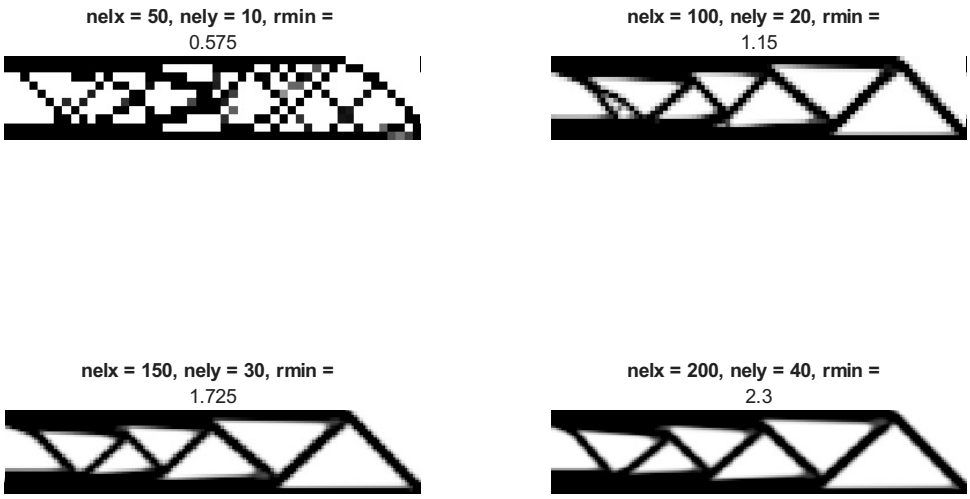


Figure 79

For a coarser mesh, the topology is more "pixelated". Some finer structural details might be lost or merged due to low resolution For a finer mesh, the boundaries becomes smooth and well-defined. The topology remains consistent with the coarser meshes, demonstrating that the filter successfully **mesh-independency**. Without scaling *rmin*, the fine and coarse meshes would have produced completely different final shapes.

– Effect on Compliance:

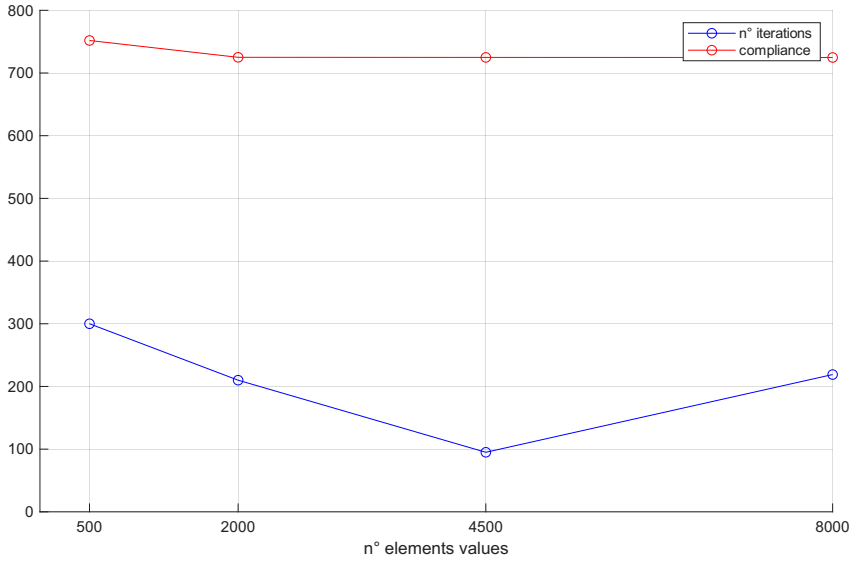


Figure 80

The compliance value tends to converge to a stable limit as the mesh is refined.

Computational Cost vs Convergence Rate:
Interestingly, coarse meshes often require more iterations to converge than the fine meshes due the lack of resolution, which causes numerical oscillations and a poorly defined sensitivity field.
However, the total computational time increases drastically with mesh refinement. This is because, the core of the Finite Element Analysis involves solving a system of linear equations (K*u = f). The computational complexity of this operation scales non-linearly with the number of degrees of freedom. Therefore, even if a 150x30 mesh requires fewer iterations than the 50x10 case, each single iteration takes lots of time to be computed.

Correlation Matrix:
Finally, a statistical correlation analysis (Pearson Correlation Matrix) was generated to roughly quantify the relationships between this input parameters and the final objective function (i.e the compliance of the structure).

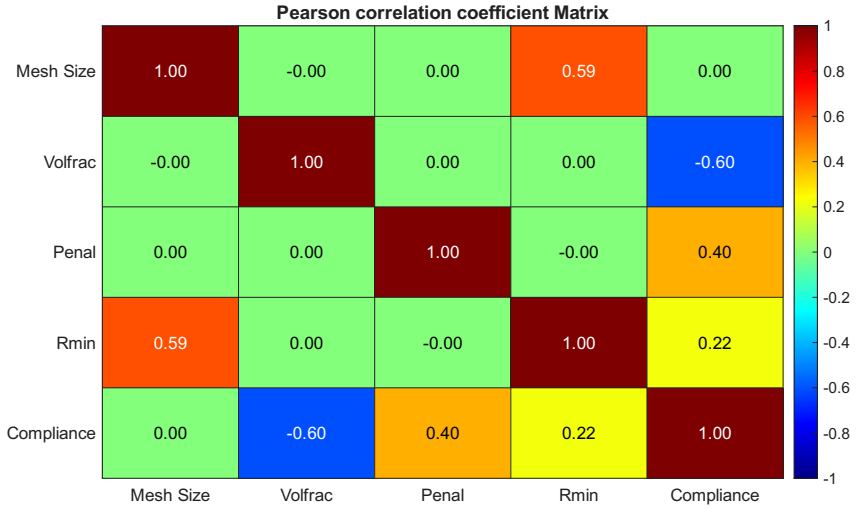


Figure 81

The values confirms the trends obtained in the previous OAT analysis:

- **Mesh Independence Validation:**
  The correlation between the Mesh Size and Compliance is exactly 0.0. This statistically confirm that the optimization result is mesh-independent.

- **Filter Scaling Artifact:**
  A positive correlation (0.59) between Rmin and the Mesh Size is observed: this is expected as the numerical filter radius (in element units) was explicitly increased with the mesh refinement.

10.2 Lower control arm Optimization on Inspire

10.2.1 Problem definition:

The aim of this section is to perform a topology optimization of the lower control arm of a McPherson suspension of a vehicle, using the software Inspire. The suspension subsystem is subjected to a Longitudinal (Fx), Lateral (Fy) and Vertical (Fz) force. The following input data represents the load scenario the lower arm is subjected to:

Parameter	Value	Units
m	390	kg
μ_x	1.2	[-]
μ_y	0.7	[-]
$Fz_{static} = m * g$	3826	N
$Fz = 2.5 * Fz_{static}$	9565	N
$Fx = \mu_x$ * Fz	11478	N
$Fy = \mu_x$ * Fz	6695	N

10.2.2 Optimization Setup:

The initial part represents a rough simplified geometry of the lower arm. The design volume is a solid large block which is connected by means of the spherical joint to the hub carrier, and by means of a couple of bushings to the chassis. The solver is told to remove material only inside the design volume itself.

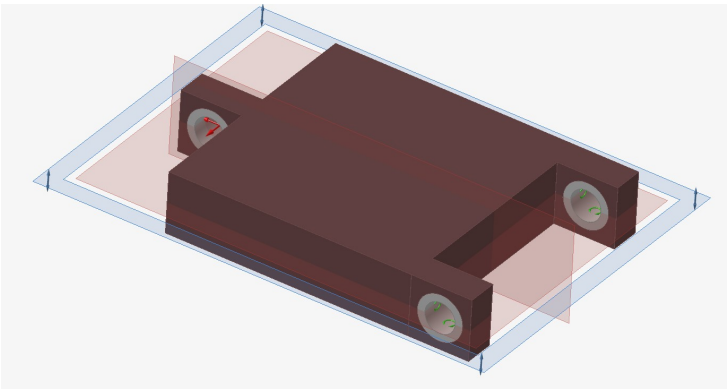


Figure 82: Initial setup

- **Loads:**
  The loads are applied on the lower arm's spherical joint attachment, they represents the reaction forces exchanged between the lower arm and the hub carrier.
  The forces are coplanar with the lower arm.

- **Supports:**
  Chassis mounts are modeled as fixed supports (Tx, Ty, Tz, Rx = 0): stiffness of

the bushings are supposed to have infinite values. The rotation of the lower arm around the bushings axis is not allowed in order to fix the structure.

- **Objective function:**
  The aim of the Topology optimization is to reduce the compliance of the structure (and so maximize the overall stiffness)

- **Design constraints:**
  - Mass target: Set to 30 %
  - Minimum thickness: imposed to prevent the solver from generating non manufacturable thin struts and to mitigate local instability (Buckling)
  - Symmetry planes: to ensure robustness against reverse loads and also to force the solver to apply the same amount of material for both the bushings "connections".
  - Manufacturing tools: a Split draw plane was applied to the designed volume to ensure manufacturability. It forces the solver to avoid undercuts so that the component can be extracted from a two-part mold (casting process)

10.2.3 Results:

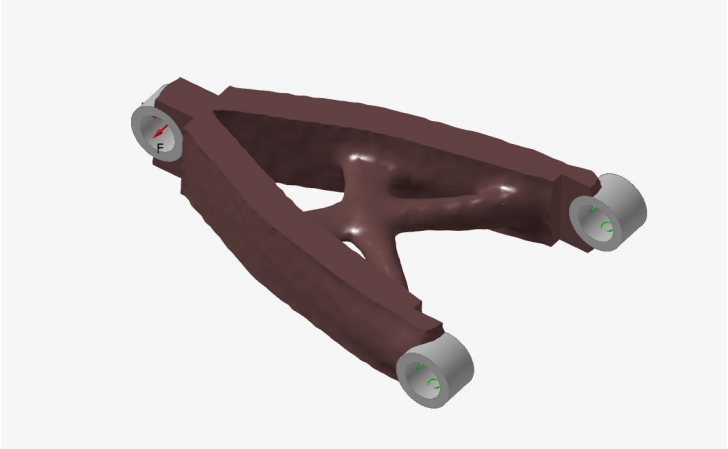
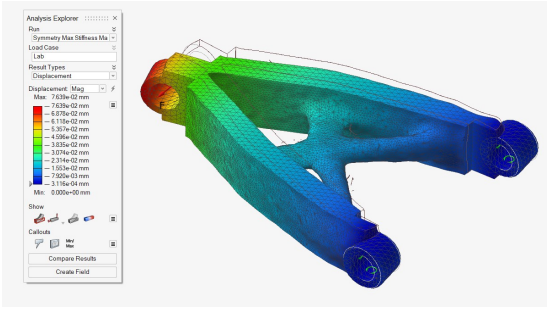


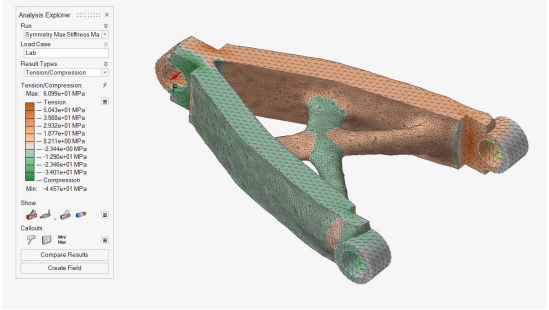
Figure 83: Optimized component

Component	**Mass [kg]**	**Volume** $[mm^3]$
Initial	19.465	2.207e6
Optimized	5.701	6.464e5

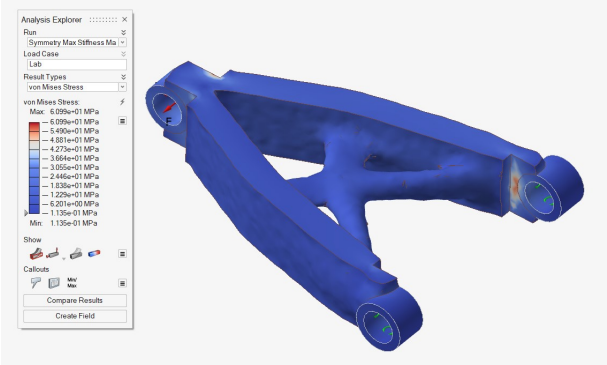
The following plots represents the Finite element analysis results performed on the optimized component:



(a) Displacement filed



(b) Compression/tension state of stress



(c) Von Mieses Stresses

The Structure is coherent with the constraints and the loads applied, and it supports perfectly the stresses (Safety factor = 6).
A final refinement of the lower arm was done by applying the polyNURBS command which helps to smooth the geometry:

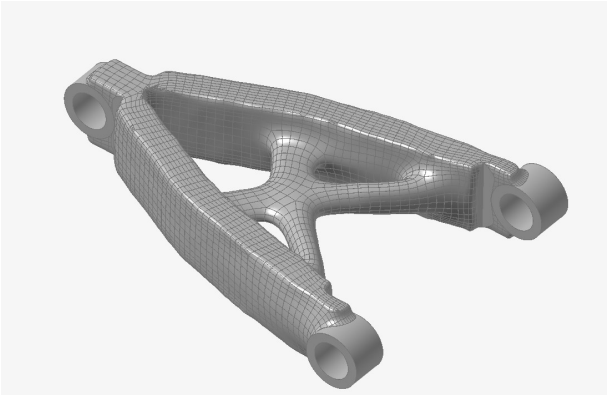


Figure 85

11 Lab RL: Suspension Optimization (Reinforcement learning)

11.1 Problem definition

The goal is to optimize the geometry of a Double Wishbone suspension system. The optimization focuses on defining the spatial configuration of the suspension's "hard points" to achieve specific kinematic targets.

Design Variables: The problem involves 9 design variables. These variables represent the spatial coordinates (x, y, z) of three critical suspension components:

- Lower Wishbone (outboard point)
- Upper Wishbone (outboard point)
- Track Rod (outboard point)

The design space is bounded, allowing the points to move within a range of ± 40 mm from their reference coordinates.

Objectives: The optimization is a 2-objective problem. The system aims to minimize the deviation from a "Target" curve for two specific kinematic behaviors during the vertical wheel travel test:

- Camber variation: minimizing the area between the actual camber curve and the target curve.
- Toe variation: minimizing the area between the actual toe curve and the target curve.

11.2 Reinforcement Learning implementation:

To solve the problem, a Reinforcement Learning algorithm was implemented: Reinforcement Learning (RL) is a machine learning paradigm created with the idea of learning through interaction with an environment. Unlike traditional Neural Networks that rely on static databases provided as training points, RL operates on a dynamic dataset generated iteratively through experience.

11.2.1 Working Principle:

The RL framework consists of two main entities: the Agent (the software/learner) and the Environment. The learning process follows a specific cycle:

1. **Observation:** at each step, the agent observes the current state of the environment
2. **Action:** based on a "policy" (the rule followed by the agent), the agent performs an action that perturbs the state of the environment
3. **Reward:** the environment provides feedback in the form of a numerical reward (or penalty) for the action performed
4. **Update:** the agent uses this new information to determine if the action was good (to be repeated) or bad (to be avoided), progressively becoming "smarter"

11.3 Reward criteria implementation

One of the main requests for this laboratory was to complete the Matlab Code which sets the RL training by implementing the Reward criteria. The hint suggested was to work on the Pareto Dominance between the result obtained in the current observation (Episode) and the ones from the previous observations.
The following algorithm was added to the function *myStepFunction.m*:

```
%% CALCULATE THE REWARD
if any(out_OF >= 99)
Reward = -100;
    else
    % Solution found at the current Episode
    current_result = out_OF(:)' ;
    if cont_prt == 0   % Condition for the first Episode
        cont_prt = 1;
        OF_prt(cont_prt, :) = current_result;
        ep_prt(cont_prt, 1) = episode;
        Reward = 1;
    else
        % Current solution comparison with actual Pareto
        current_Pareto = OF_prt(1:cont_prt, :);

        % current solution doesn't dominate any points of the Pareto
        if any(all(current_Pareto <= current_result, 2) &   any(
   current_Pareto < current_result, 2))
            Reward = -1;
        % Current solution dominates at least one point of the Pareto
        else
            % Clean the Pareto from dominated solutions
            dominated = all(current_result <= current_Pareto, 2) & any(
   current_result < current_Pareto, 2);
            n = sum(dominated);

            if n > 0
                keep = ~dominated; % Keep the non-dominated points
                n_survivors = sum(keep);

                % Reconstruct the Pareto
                OF_prt(1:n_survivors, :) = current_Pareto(keep, :);
                ep_prt(1:n_survivors) = ep_prt(keep);
                OF_prt(n_survivors+1:cont_prt, :) = inf;
                ep_prt(n_survivors+1:cont_prt) = inf;
                cont_prt = n_survivors;
            end

            % I add the current solution to the Pareto set
            cont_prt = cont_prt + 1;
            OF_prt(cont_prt, :) = current_result;
            ep_prt(cont_prt, 1) = episode;
            Reward = 1;
        end
    end
end
```

In addition, a *paretosort* function was implemented to compute the Pareto Optimal Set, the algorithm is an adapted version of the Sorting Algorithm used in previous laboratories.

11.4 Analysis of RL Optimization Result:

The results obtained using the MORL-DB algorithm for the suspension hardpoints optimization are analyzed regarding training convergence, the resulting Pareto front, and kinematic validation.

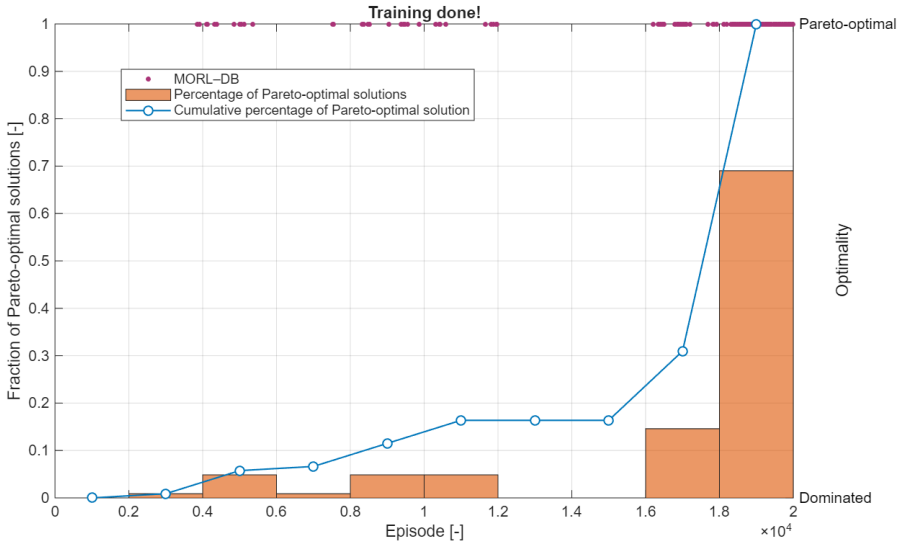


Figure 86

As shown in the (Fig. 86), the training process exhibits two distinct phases. Up to approximately 1.5×10^4 episodes, the agent operates in an exploration regime, identifying relatively few optimal solutions. In the final phase (from 1.6×10^4 episodes onwards), a sharp increase in the cumulative percentage of Pareto-optimal solutions is observed, indicating that the agent has successfully learned to maximize the reward and the algorithm has reached convergence.

Along the Pareto front, three points are chosen.

- **Two Extreme Points:** The solutions at the boundaries of the front minimize a single objective at the expense of the other.
- **Knee Point:** The intermediate solution.

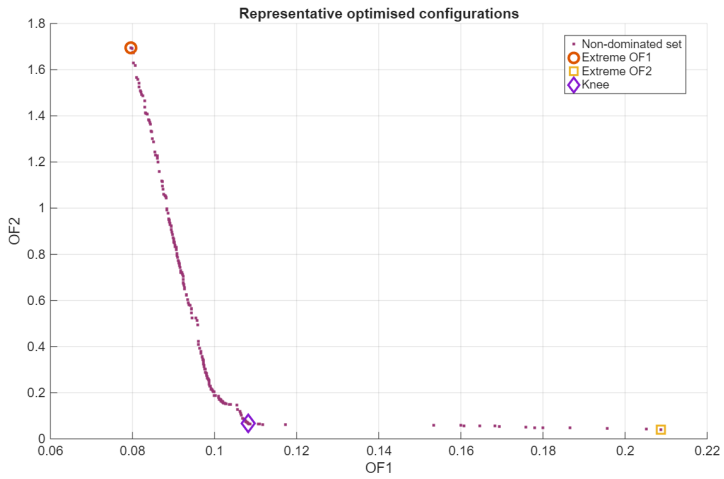


Figure 87

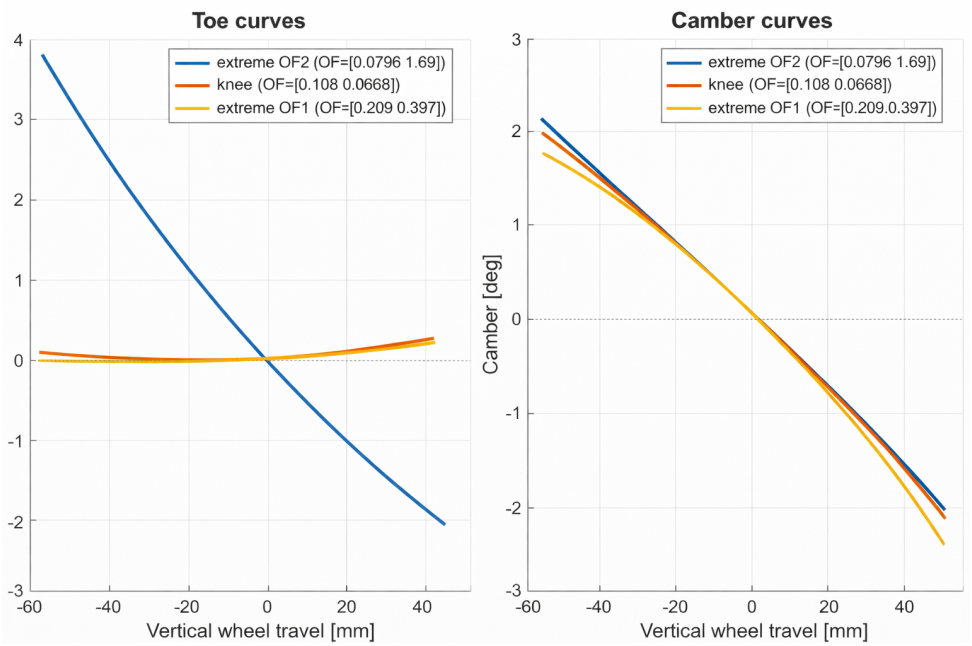


Figure 88

Analyzing Toe and Camber curves in (Fig. 88):

- The **Extreme OF1** solution (yellow line) ensures a perfectly flat Toe curve (minimal bump steer, Low OF2), but its Camber recovery deviates from the ideal target (High OF1).
- The **Extreme OF2** solution (blue line) minimizes Camber error but exhibits the worst Toe behavior, with a steep curve ("vertical") indicating significant bump steer (High OF2).
- The **Knee** solution (orange line) offers the optimal balance: it maintains a very low Toe variation (close to the yellow line) while preserving a Camber recovery curve very similar to the optimal blue one.